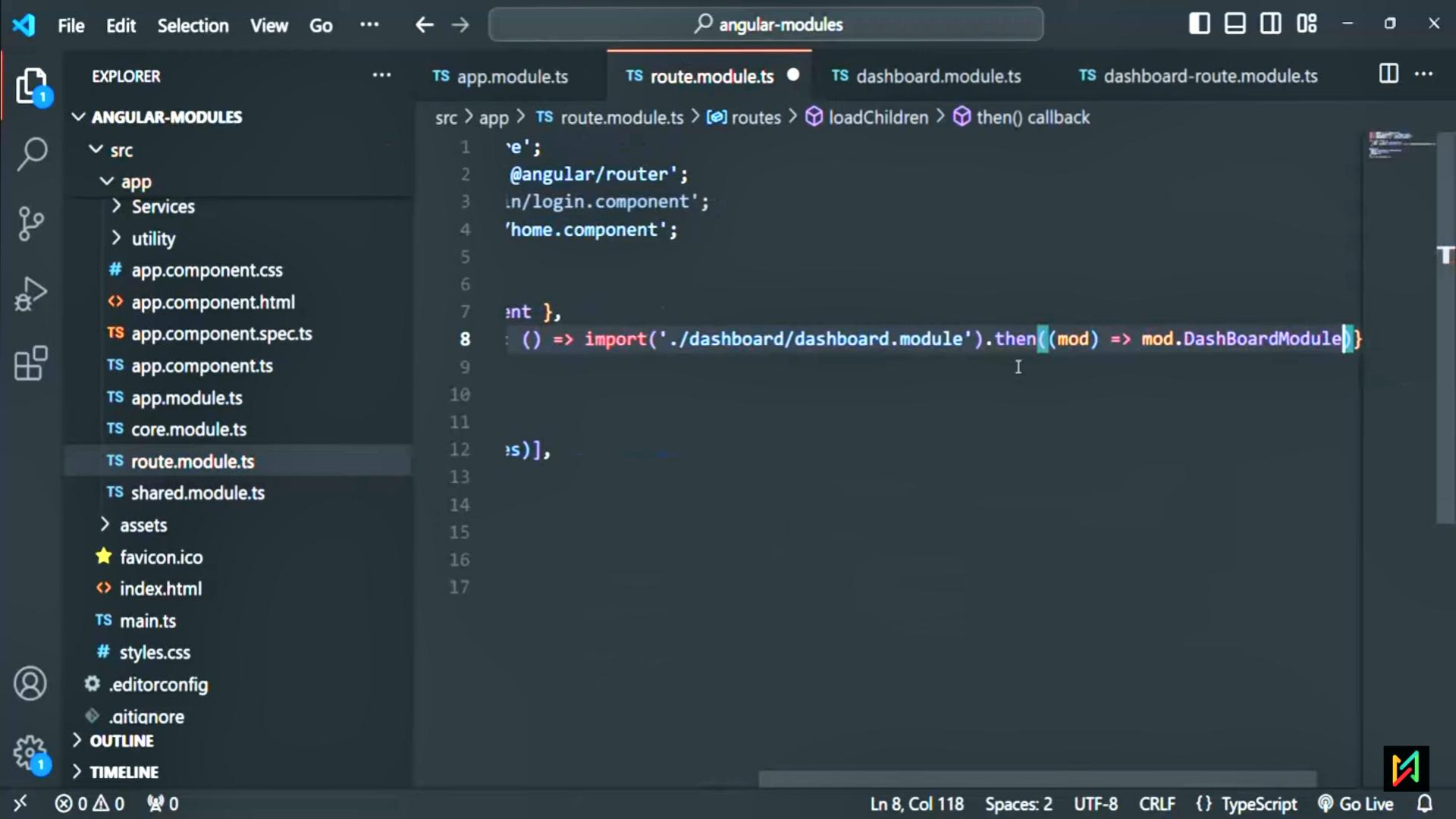
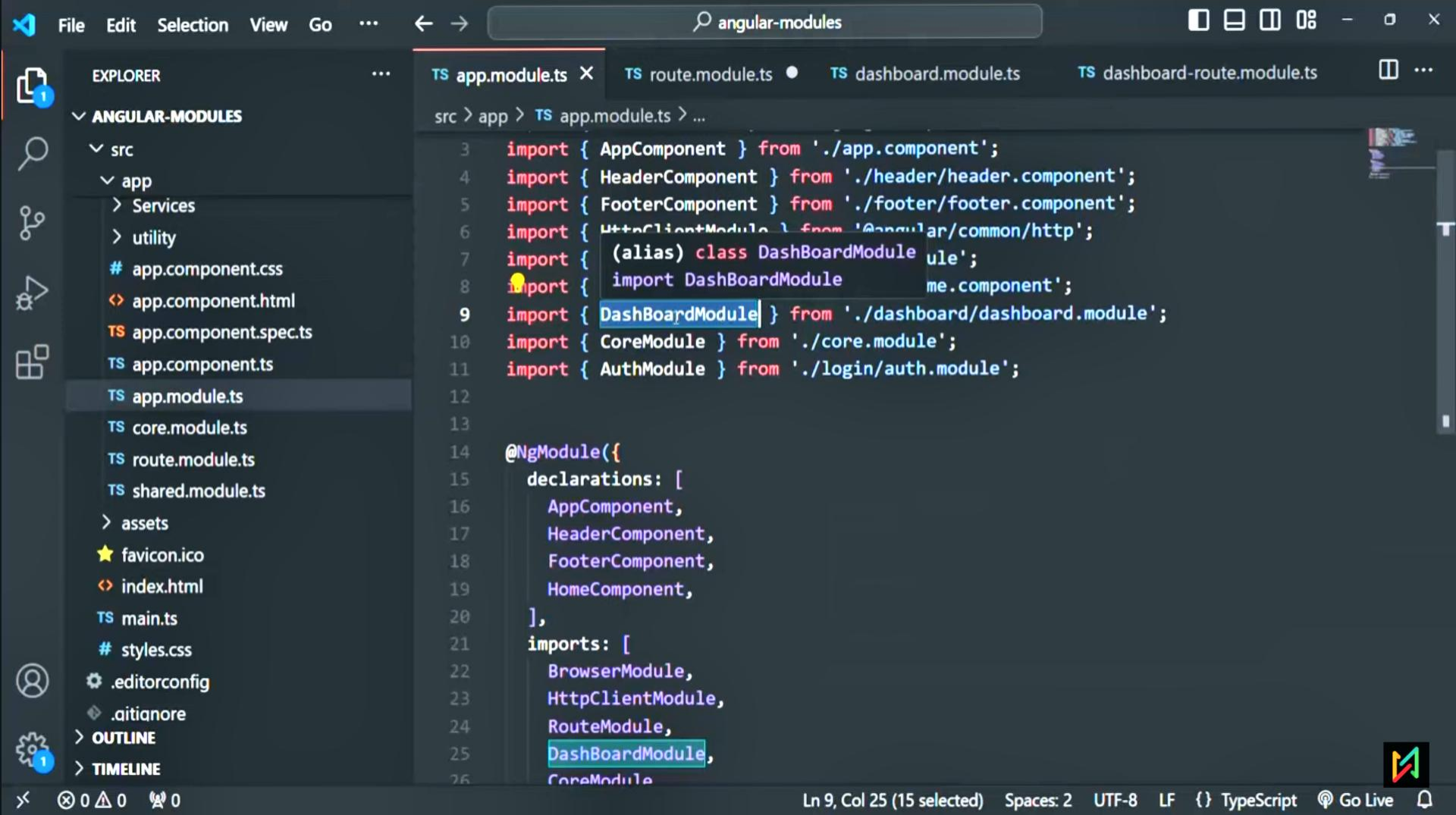






EXPLORER

ANGULAR-ROUTING

- > Models
- > not-found
- > Services
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts
- TS app.component.ts
- TS app.module.ts
- TS auth.guard.ts
- TS routing.module.ts
- > assets
- ★ favicon.ico
- <> index.html
- TS main.ts
- # styles.css
- ⚙ .editorconfig
- 📄 .gitignore
- { } angular.json
- > OUTLINE
- > TIMELINE

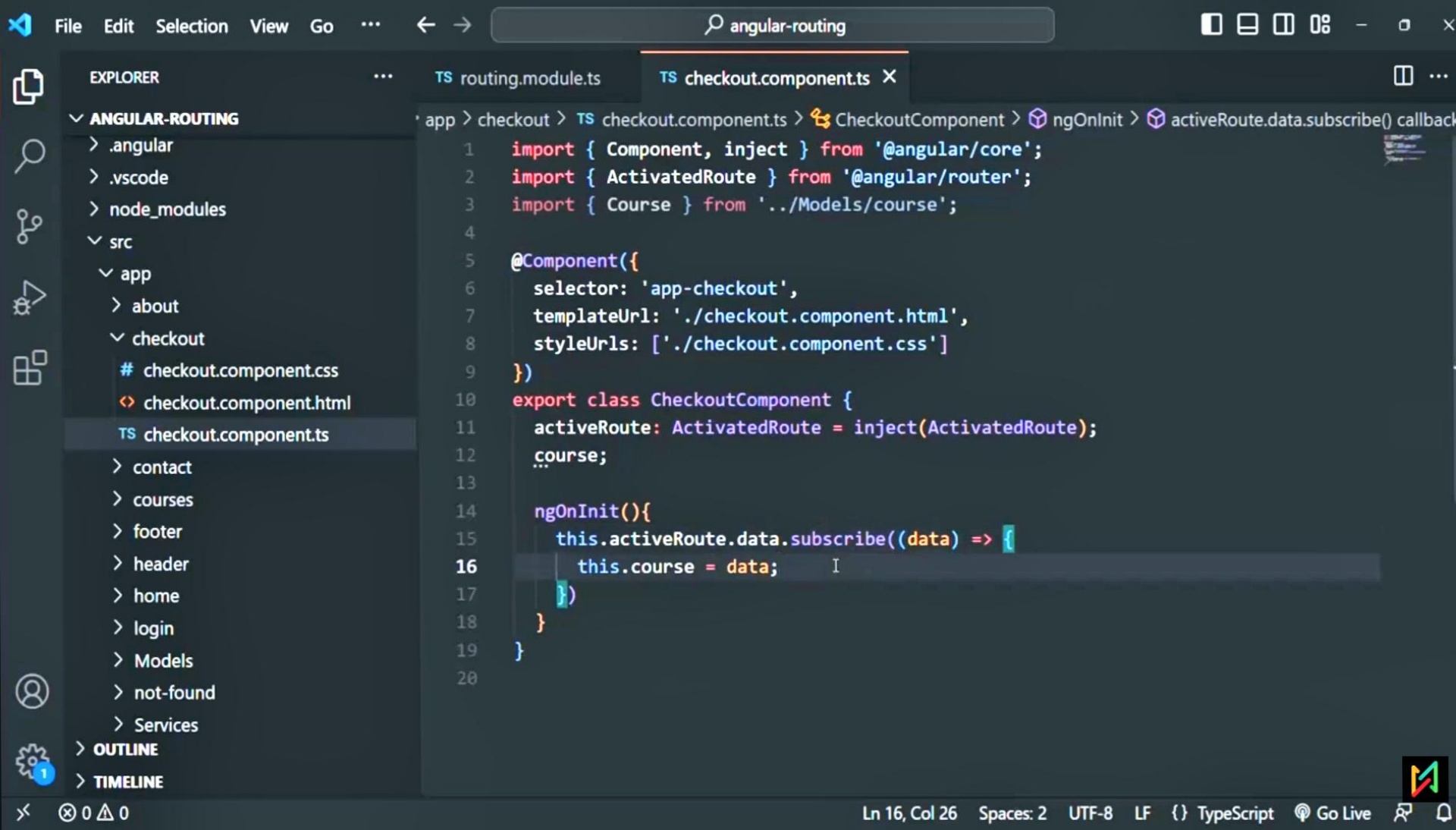
TS routing.module.ts

src > app > TS routing.module.ts > routes > children > path

```

14
15
16 //DEFINE ROUTE
17 const routes: Routes = [
18
19     {path: '', component: HomeComponent},
20     {path: 'Home', component: HomeComponent},
21     {path: 'About', component: AboutComponent},
22     {path: 'Contact', component: ContactComponent, canDeactivate: [(comp: ContactComponent) => {return true;}]},
23     {path: 'Courses', component: CoursesComponent, resolve: {courses: resolve}},
24     {path: 'Courses', canActivateChild: [CanActivateChild], children: [
25         {path: 'Course/:id', component: CourseDetailComponent},
26         {path: 'Popular', component: PopularComponent},
27         {path: 'Checkout', component: CheckoutComponent, data: {name: 'Test Course'}},
28     ]},
29     {path: 'Login', component: LoginComponent},
30     {path: '**', component: NotFoundComponent},
31 ]
32
33 @NgModule({
34     imports: [
35         RouterModule.forRoot(routes, {enableTracing: true})
36     ],
37     exports: [RouterModule]
38 })

```



EXPLORER

TS routing.module.ts

TS checkout.component.ts X

ANGULAR-ROUTING

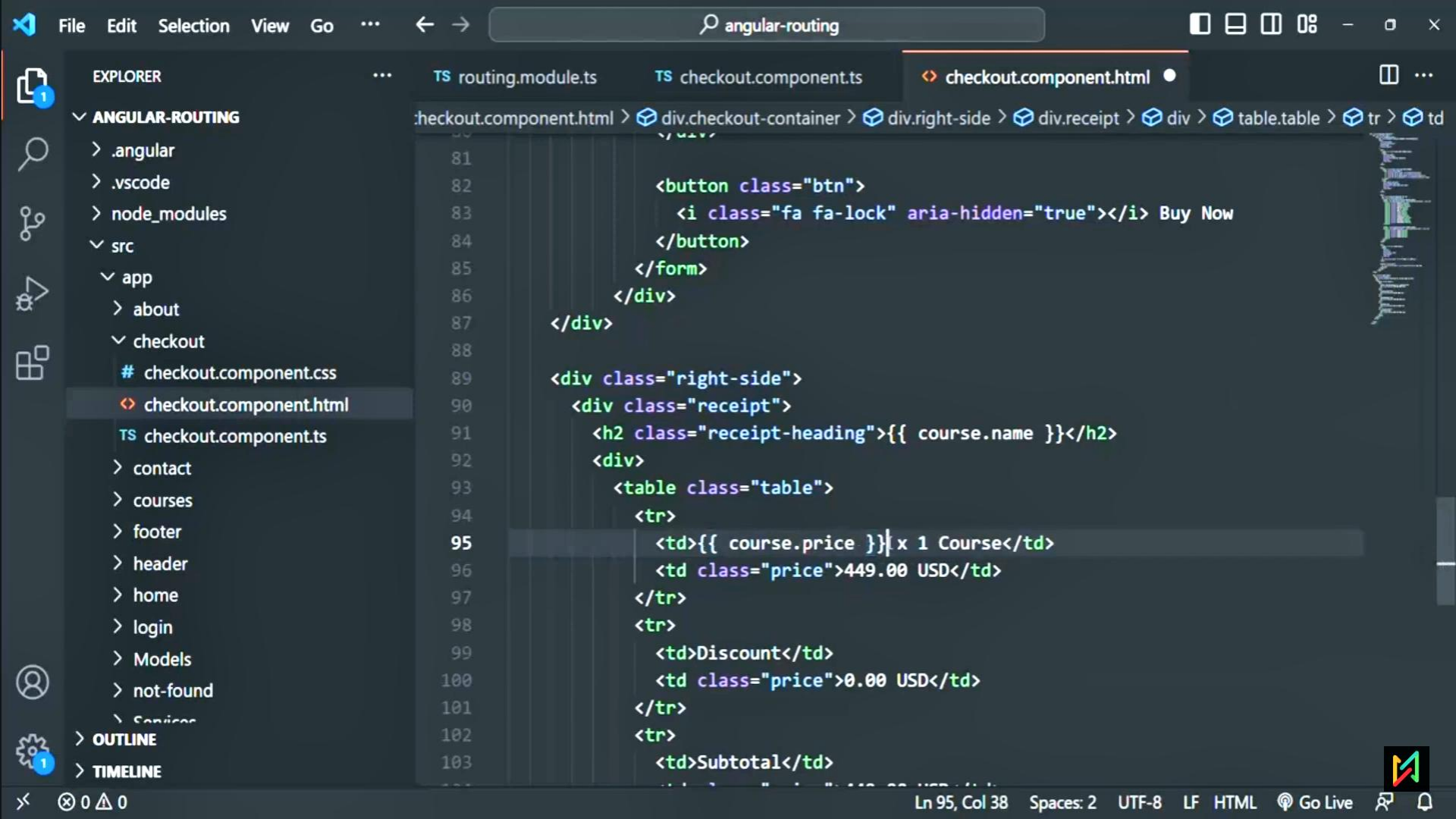
- > .angular
- > .vscode
- > node_modules
- > src
 - > app
 - > about
 - > checkout
 - # checkout.component.css
 - <> checkout.component.html
 - TS checkout.component.ts
 - > contact
 - > courses
 - > footer
 - > header
 - > home
 - > login
 - > Models
 - > not-found
 - > Services

> OUTLINE

> TIMELINE

app > checkout > TS checkout.component.ts > CheckoutComponent > ngOnInit > activeRoute.data.subscribe() callback

```
1 import { Component, inject } from '@angular/core';
2 import { ActivatedRoute } from '@angular/router';
3 import { Course } from '../Models/course';
4
5 @Component({
6   selector: 'app-checkout',
7   templateUrl: './checkout.component.html',
8   styleUrls: ['./checkout.component.css']
9 })
10 export class CheckoutComponent {
11   activeRoute: ActivatedRoute = inject(ActivatedRoute);
12   course;
13
14   ngOnInit(){
15     this.activeRoute.data.subscribe((data) => {
16       this.course = data;
17     })
18   }
19 }
20
```

File

Edit

Selection

View

Go

...

←

→

angular-routing

1

EXPLORER

...

TS routing.module.ts

courses.component.html

TS checkout.component.ts

checkout.compone

...

ANGULAR-ROUTING

iv.courses-page-all-courses > div.courses-page-course-list > div.course-card > div.course-action-buttons > button.btn

checkout.component.html

TS checkout.component.ts

> contact

> courses

> course-detail

courses.component.css

courses.component.html

TS courses.component.ts

> footer

> header

> home

> login

> Models

> not-found

> Services

app.component.css

app.component.html

TS app.component.spec.ts

> OUTLINE

> TIMELINE

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

<div class="course-card" *ngFor="let course of AllCourses">

<div class="course-image">

</div>

<div class="course-title">

{{ course.title }}

</div>

<div class="course-desc">

{{ course.desc.substring(0, 120) }}

</div>

<div class="course-price-rating">

<div class="course-price">PRICE: {{ course.price | currency }}

<div class="course-rating">RATINGS: {{ course.rating }}

</div>

</div>

<div class="course-action-buttons">

<button class="btn" routerLink="/Courses/Checkout" [state]="course">

<button class="btn" [routerLink]="'/Courses/Course/' + course.id">DET

</div>

</div>

</div>

div>

routerLink="/Courses" [queryParams]="{search: 'JavaScript'}">Show JavaScript Cour

Ln 26, Col 88 (16 selected)

Spaces: 4

UTF-8

LF

HTML

Go Live

EXPLORER

ANGULAR-ROUTING

- checkout.component.html
- TS checkout.component.ts
- > contact
- > courses
 - > course-detail
 - # courses.component.css
 - <> courses.component.html
 - TS courses.component.ts
- > footer
- > header
- > home
- > login
- > Models
- > not-found
- > Services
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts

OUTLINE

TIMELINE

src > app > checkout > TS checkout.component.ts > CheckoutComponent > course

```

1  import { Component, inject } from '@angular/core';
2  import { ActivatedRoute, Router } from '@angular/router';
3  import { Course } from '../Models/course';
4
5  @Component({
6    selector: 'app-checkout',
7    templateUrl: './checkout.component.html',
8    styleUrls: ['./checkout.component.css']
9  })
10 export class CheckoutComponent {
11   activeRoute: ActivatedRoute = inject(ActivatedRoute);
12   router: Router = inject(Router);
13   course;
14
15   ngOnInit(){
16     // this.activeRoute.data.subscribe((data) => {
17     //   this.course = data;
18     // })
19
20     this.course = this.router.getCurrentNavigation().extras.state;
21   }
22 }

```


Angular Routing

localhost:4200/Courses/Checkout

SEARCH POPULAR TESTIMONY SERVICES CONTACT

procademy

HOME ABOUT CONTACT COURSES

Payment Information

Full Name

Satoshi Nakamoto

Card Number

1111-2222-3333-4444

Expires on:

Month

Month: ▾

Year

Year ▾

CVV

415

[What is CVV?](#)

Buy Now

Discount

0.00 USD

Subtotal

Tax

47.41 USD

Total

Update

2 2

Elements Console

top Filter 1 hidden

5 Issues: 2 3

Router Event: NavigationEnd router.mjs:6502

NavigationEnd(id: 5, url: '/Courses/Checkout', urlAfterRedirects: '/Courses/Checkout') router.mjs:6503

NavigationEnd router.mjs:6504

ERROR TypeError: Cannot read properties of null (reading 'extras') core.mjs:10920

at CheckoutComponent.ngOnInit (checkout.component.ts:20:53)

at callHookInternal (core.mjs:3940:14)

at callHook (core.mjs:3967:13)

at callHooks (core.mjs:3922:17)

at executeInitAndCheckHooks (core.mjs:3872:9)

at refreshView (core.mjs:12913:21)

at detectChangesInView (core.mjs:13062:9)

at detectChangesInEmbeddedViews (core.mjs:13006:13)

at refreshView (core.mjs:12922:9)

at detectChangesInView (core.mjs:13062:9)

Router Event: Scroll router.mjs:6502

Scroll(anchor: 'null', position: 'null') router.mjs:6503

Scroll router.mjs:6504

ERROR TypeError: Cannot read properties of undefined (reading 'name') core.mjs:10920

at CheckoutComponent_Template (checkout.component.html:91:37)

at ReactiveViewConsumer.runInContext (core.mjs:11103:13)

at executeTemplate (core.mjs:11404:22)

at refreshView (core.mjs:12898:13)

at detectChangesInView (core.mjs:13062:9)

at detectChangesInComponent (core.mjs:13038:5)



EXPLORER

ANGULAR-ROUTING

- checkout.component.html
- TS checkout.component.ts
- > contact
- > courses
 - > course-detail
 - # courses.component.css
 - <> courses.component.html
 - TS courses.component.ts
- > footer
- > header
- > home
- > login
- > Models
- > not-found
- > Services
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts

> OUTLINE

> TIMELINE

TS routing.module.ts

<> courses.component.html

TS checkout.component.ts

<> checkout.compone

src > app > checkout > TS checkout.component.ts > CheckoutComponent > ngOnInit

```
1 import { Component, inject } from '@angular/core';
2 import { ActivatedRoute, Router } from '@angular/router';
3 import { Course } from '../Models/course';
4
5 @Component({
6   selector: 'app-checkout',
7   templateUrl: './checkout.component.html',
8   styleUrls: ['./checkout.component.css']
9 })
10 export class CheckoutComponent {
11   activeRoute: ActivatedRoute = inject(ActivatedRoute);
12   router: Router = inject(Router);
13   course;
14
15   ngOnInit(){
16     // this.activeRoute.data.subscribe((data) => {
17     //   this.course = data;
18     // })
19
20     //this.course = this.router.getCurrentNavigation().extras.state;
21     this.course = history.state;
22   }
23 }
24
```

Payment Information

Full Name

Satoshi Nakamoto

Card Number



1111-2222-3333-4444

Expires on:

Month

Month:

Year

Year

CVV

415

[What is CVV?](#)

Buy Now

599 x 1 Course

599 USD

Discount

0.00 USD

Subtotal

599 USD

Tax

47.41 USD

Total

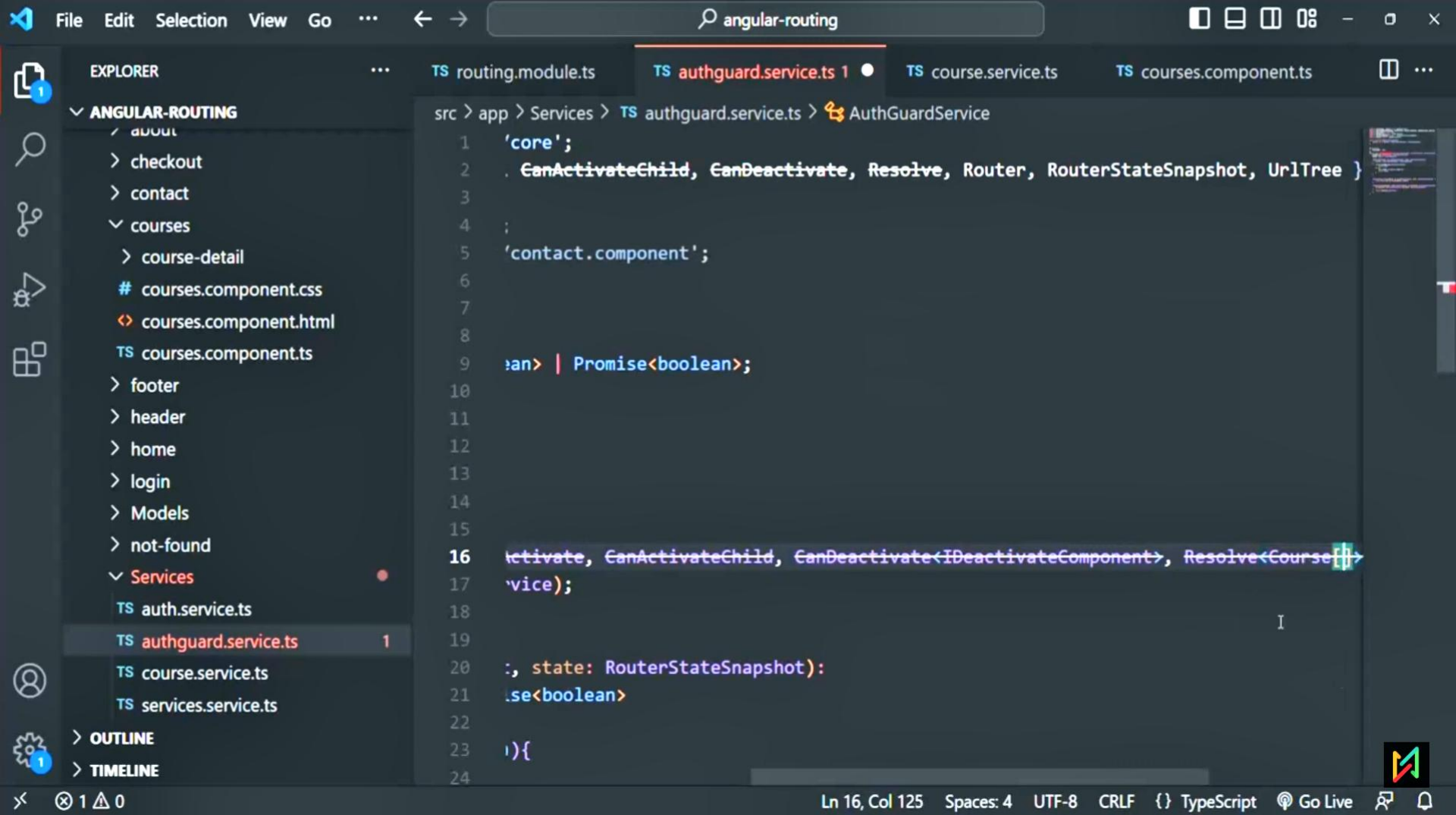
646.41



Resolve Route Guard

Resolve route guard in Angular can be used when we want to load some data before we navigate to a route.





EXPLORER

ANGULAR-ROUTING

- about
- checkout
- contact
- courses
 - course-detail
 - courses.component.css
 - courses.component.html
 - courses.component.ts
- footer
- header
- home
- login
- Models
- not-found
- Services
 - auth.service.ts
 - authguard.service.ts
 - course.service.ts
 - services.service.ts

OUTLINE

TIMELINE

TS routing.module.ts

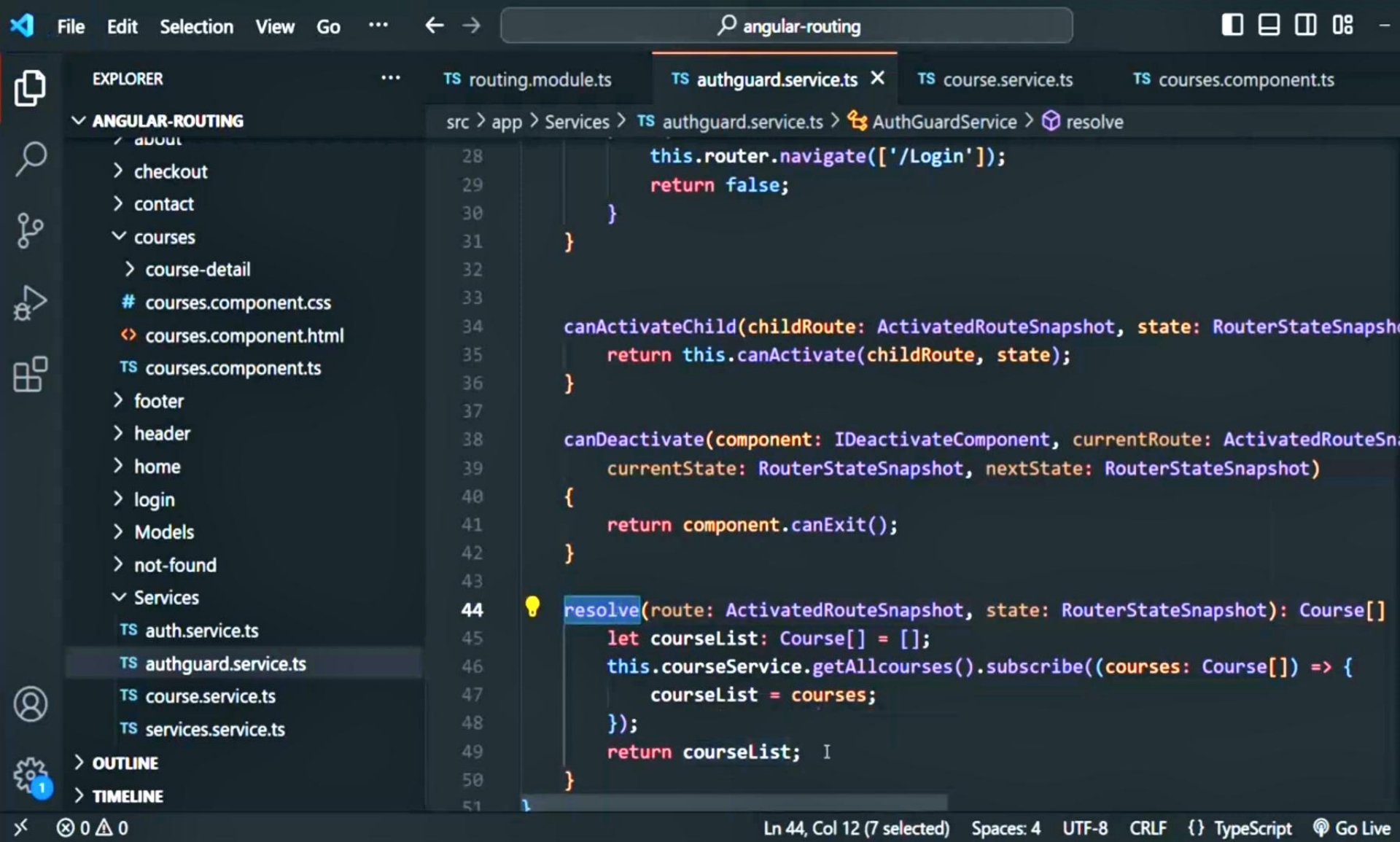
TS authguard.service.ts 1

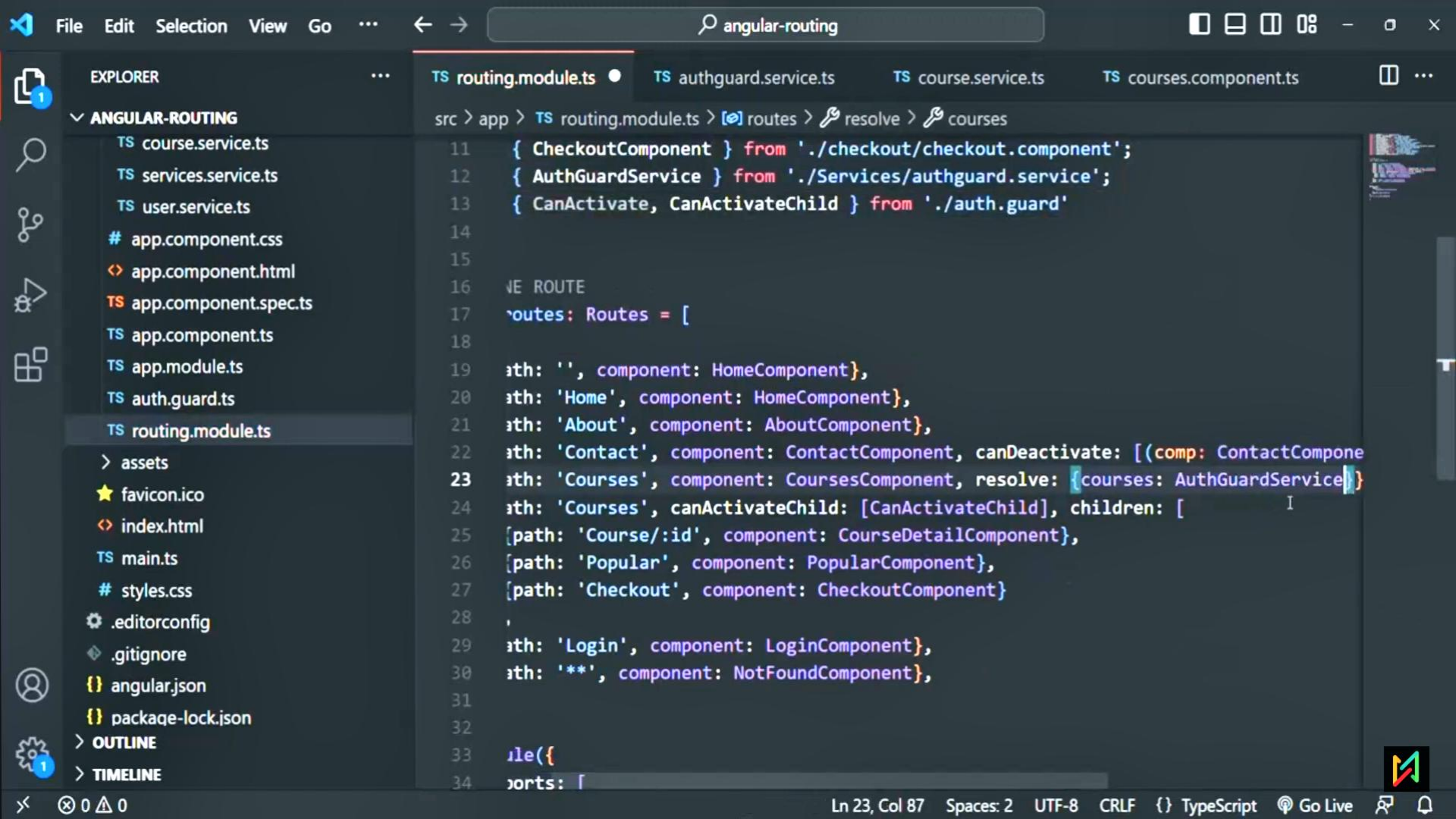
TS course.service.ts

TS courses.component.ts

src > app > Services > TS authguard.service.ts > AuthGuardService

```
1 'core';
2 . CanActivateChild, CanDeactivate, Resolve, Router, RouterStateSnapshot, UrlTree }
3
4 ;
5 'contact.component';
6
7
8
9 can > | Promise<boolean>;
10
11
12
13
14
15
16 canActivate, CanActivateChild, CanDeactivate<IDeactivateComponent>, Resolve<Course>
17 vice);
18
19
20 :, state: RouterStateSnapshot):
21 use<boolean>
22
23 ){}
24
```





courses > TS courses.component.ts > CoursesComponent > ngOnInit > `activeRoute.queryParamMap.subscribe()` callback

> TIMELINE

```

11 export class CoursesComponent implements OnInit{
12   coursesService = inject(CourseService);
13   AllCourses: Course[];
14   searchString: string;
15
16   activeRoute: ActivatedRoute = inject(ActivatedRoute);
17
18   ngOnInit(){
19     // this.searchString = this.activeRoute.snapshot.queryParams['search'];
20     // this.searchString = this.activeRoute.snapshot.queryParamMap.get('search');
21     // console.log(this.searchString);
22
23     this.activeRoute.queryParamMap.subscribe((data) => {
24       this.searchString = data.get('search');
25
26       if(this.searchString === undefined || this.searchString === '' || this.searchString === null){
27         // this.coursesService.getAllcourses().subscribe((data: Course[]) => {
28         //   this.AllCourses = data;
29         // });
30
31         this.AllCourses = this.activeRoute.snapshot.data['courses'];
32       }else{
33         this.AllCourses = this.coursesService.courses
34           .filter(x => x.title.toLowerCase()

```


File

Edit

Selection

View

Go

...

←

→

angular-routing

EXPLORER

...

TS routing.module.ts

TS authguard.service.ts X

TS course.service.ts

TS courses.component.ts

▼ ANGULAR-ROUTING

TS courses.component.ts

> footer

> header

> home

> login

> Models

> not-found

▼ Services

TS auth.service.ts

TS authguard.service.ts

TS course.service.ts

TS services.service.ts

TS user.service.ts

app.component.css

<> app.component.html

TS app.component.spec.ts

TS app.component.ts

TS app.module.ts

TS auth.guard.ts

> OUTLINE

> TIMELINE

src > app > Services > TS authguard.service.ts > AuthGuardService > resolve

27

28

29

30

31

32

33

34 te: RouterStateSnapshot): boolean | UriTree | Observable<boolean | UriTree> | Prom

35

36

37

38 ute: ActivatedRouteSnapshot,

39 StateSnapshot)

40

41

42

43

44 eSnapshot): Course[] | Observable<Course[]> | Promise<Course[]> {

45

46 s: Course[]]) => {

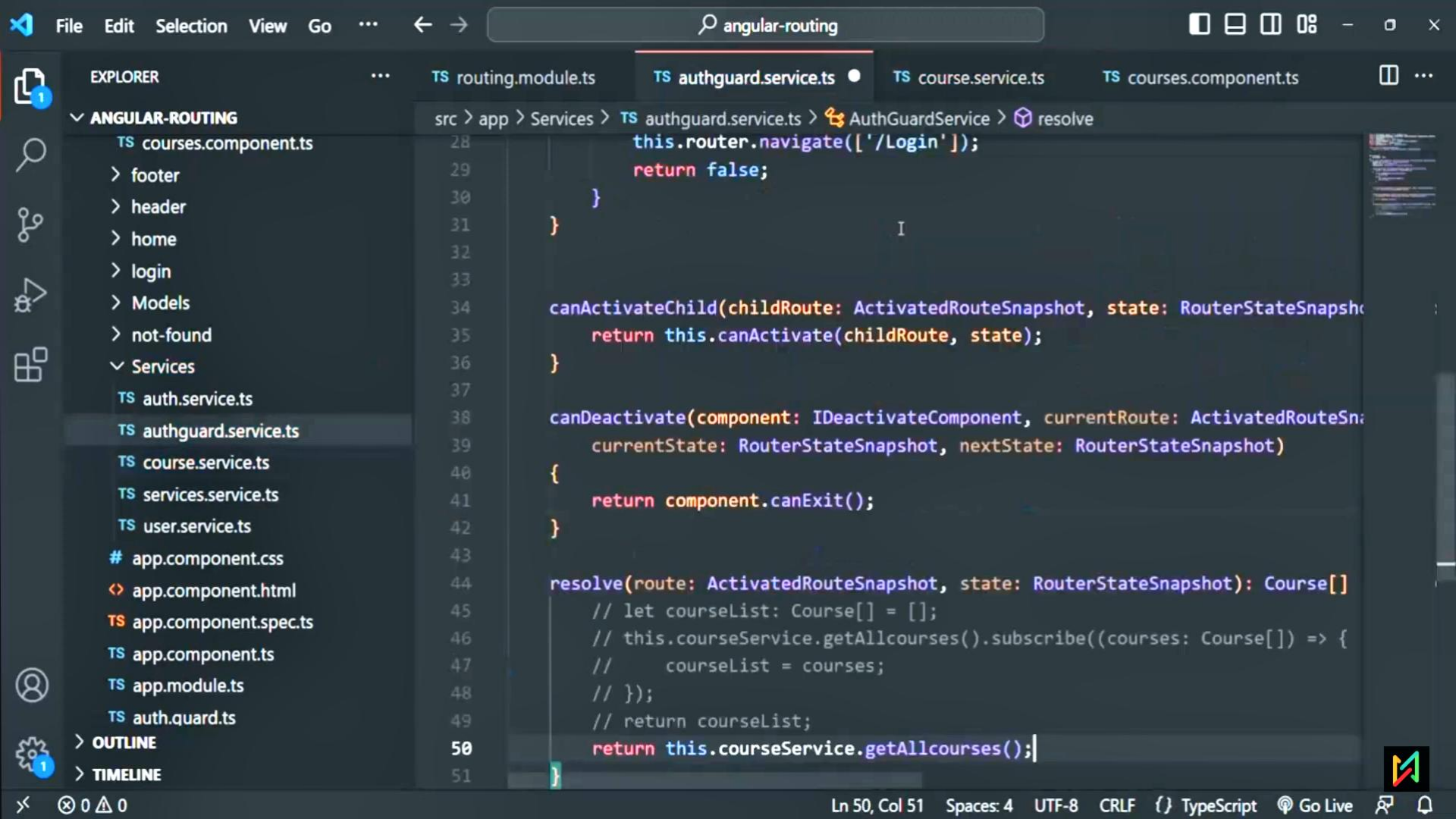
47

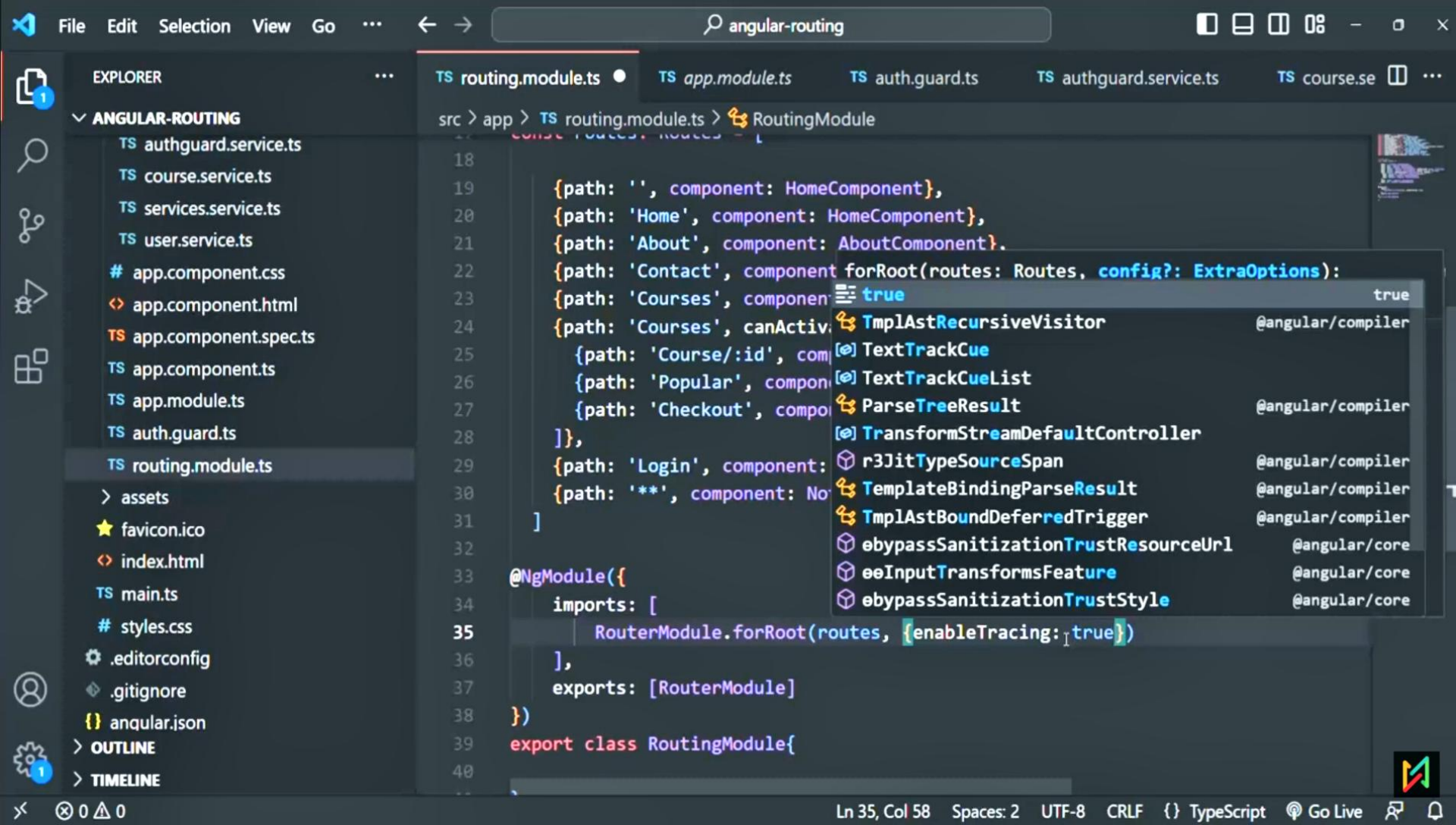
48

49

50

Ln 44, Col 124 (17 selected) Spaces: 4 UTF-8 CRLF {} TypeScript Go Live





Angular Routing

localhost:4200/Contact

f

t

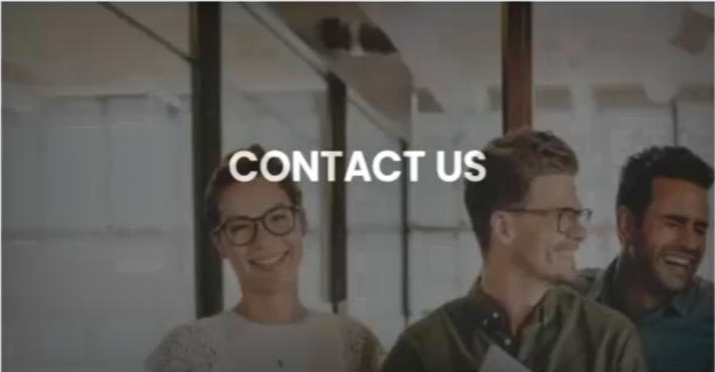
+

in

SEARCHPOPULARTESTIMONYSERVICESCONTACT

procademy

HOMEABOUTCONTACTCOURSES



CONTACT US

Contact us for more details

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

4 Issues: 3 1

path: '') { Route(url: 'Contact', path: 'Contact') })

ResolveStart {id: 2, url: '/Contact', urlAfterRedirects: '/Contact', state: RouterStateSnapshot, type: 5}

Router Event: ResolveEnd

ResolveEnd(id: 2, url: '/Contact', urlAfterRedirects: '/Contact', state: Route(url: '', path: '') { Route(url: 'Contact', path: 'Contact') })

ResolveEnd {id: 2, url: '/Contact', urlAfterRedirects: '/Contact', state: RouterStateSnapshot, type: 6}

Router Event: ActivationEnd

ActivationEnd(path: 'Contact')

ActivationEnd {snapshot: ActivatedRouteSnapshot, type: 14}

Router Event: ChildActivationEnd

ChildActivationEnd(path: '')

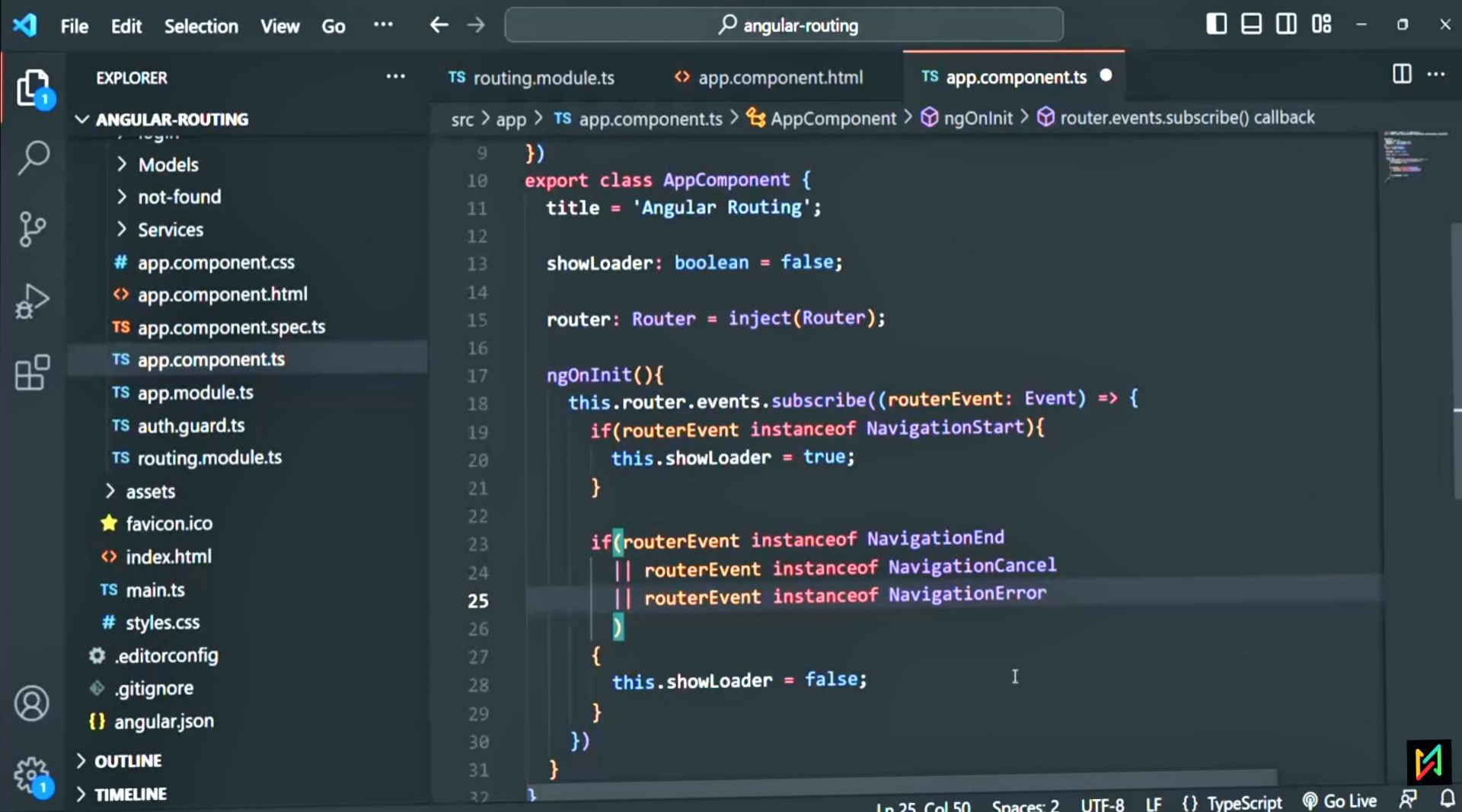
ChildActivationEnd {snapshot: ActivatedRouteSnapshot, type: 12}

Router Event: NavigationEnd

Router Event: Scroll

Scroll(anchor: 'null', position: 'null')

Scroll {routerEvent: NavigationEnd, position: null, anchor: null, type: 15}



What is Route Guard

Angular route guards can be used to control whether the user can navigate to or away from a route based on a given condition.



Why Route Guard

Allowing the users to navigate to all parts of the application is not a good idea. And we need to restrict the use from accessing certain routes, until the user performs a specific action like login to application. So, you can use route guards for following scenario's

- Restrict a use from accessing a route.
- Ask user to save changes before moving away from view.
- Validating the route parameters before navigating to the route.
- Fetch some data before you display component view of a route.



Type of Route Guard

CanActivate

This guard decides if a route can be accessed by a user or not. This guard is useful in the circumstance where the user is not authorized to navigate to the target component.

CanActivateChild

This guard decides, if a user can leave a route or not. This guard is useful in case where the user might have some pending changes, which was not saved.

CanDeactivate

This guard determines whether a child route can be activated or not.

Type of Route Guard

Resolve

This guard delays the activation of the route until some tasks are complete. You can use the guard to pre-fetch the data from the backend API, before activating the route

CanLoad

The **CanLoad** Guard prevents the loading of the Lazy Loaded Module. We generally use this guard when we do not want to unauthorized user to be able to even see the source code of the module.



Route Guard Implementation

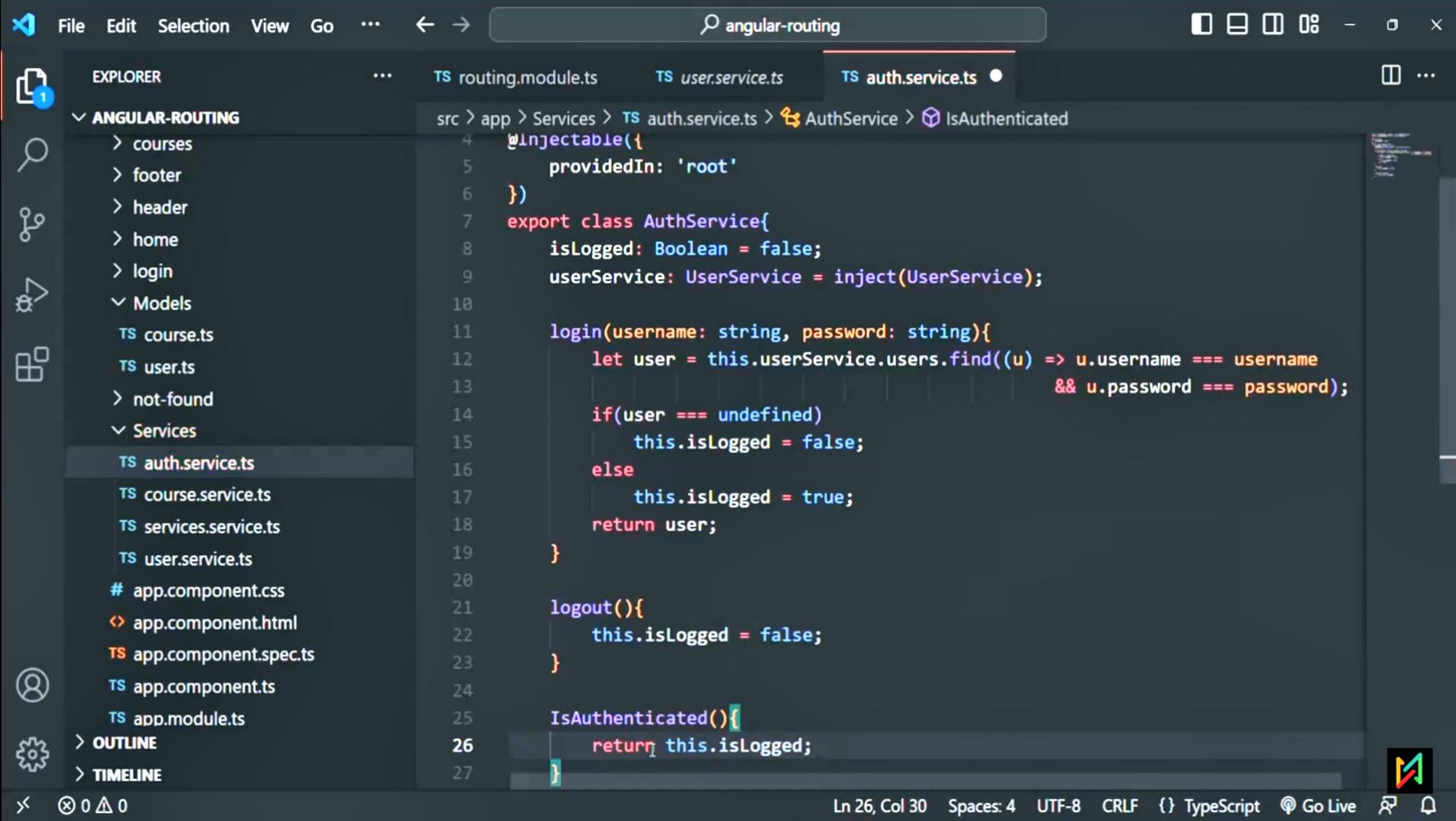
Angular 14 & Lower Versions

1. Create a service which inherits from specific route guard interface.
2. Implement the method provided by that route guard interface.
3. Return a boolean value / data from that method.
4. Assign the service to route guard property of route object.

Angular 15 & Higher Versions

1. Create a function which should return a boolean value / data.
2. Assign that function to specific route guard property of route object.





EXPLORER

ANGULAR-ROUTING

> courses

> footer

> header

> home

> login

Models

TS course.ts

TS user.ts

> not-found

Services

TS auth.service.ts

TS course.service.ts

TS services.service.ts

TS user.service.ts

app.component.css

<> app.component.html

TS app.component.spec.ts

TS app.component.ts

TS app.module.ts

> OUTLINE

> TIMELINE

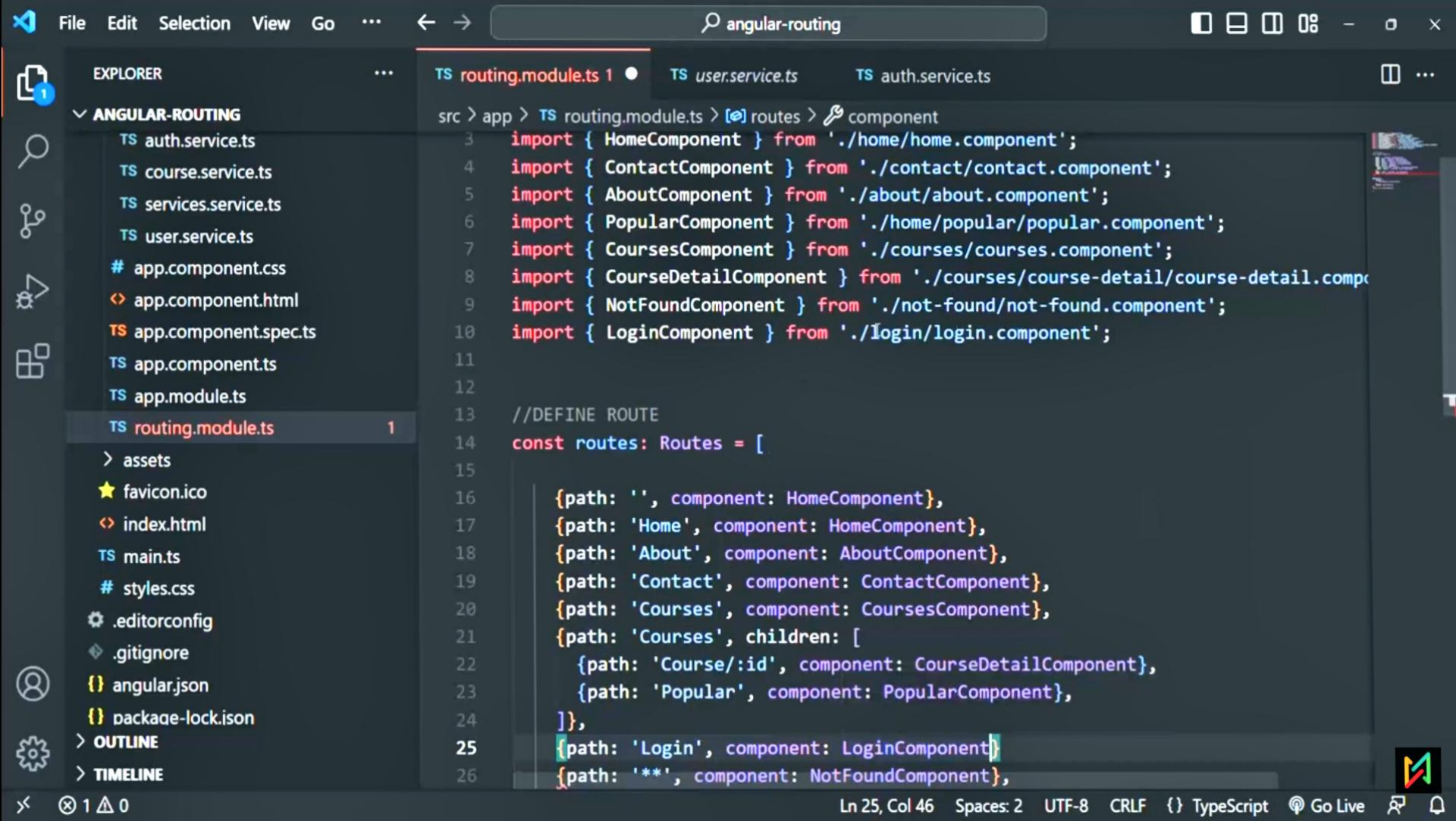
TS routing.module.ts

TS user.service.ts

TS auth.service.ts

src > app > Services > TS auth.service.ts > AuthService > IsAuthenticated

```
4  @Injectable({
5      providedIn: 'root'
6  })
7  export class AuthService{
8      isLoggedIn: Boolean = false;
9      userService: UserService = inject(UserService);
10
11      login(username: string, password: string){
12          let user = this.userService.users.find((u) => u.username === username
13                                                         && u.password === password);
14          if(user === undefined)
15              this.isLoggedIn = false;
16          else
17              this.isLoggedIn = true;
18          return user;
19      }
20
21      logout(){
22          this.isLoggedIn = false;
23      }
24
25      isAuthenticated(){
26          return this.isLoggedIn;
27      }
```



EXPLORER

▼ ANGULAR-ROUTING

TS auth.service.ts
TS course.service.ts
TS services.service.ts
TS user.service.ts
app.component.css
<> app.component.html
TS app.component.spec.ts
TS app.component.ts
TS app.module.ts
TS routing.module.ts 1

> assets
★ favicon.ico
<> index.html
TS main.ts
styles.css
⚙️ .editorconfig
📁 .gitignore
{ } angular.json
{ } package-lock.json

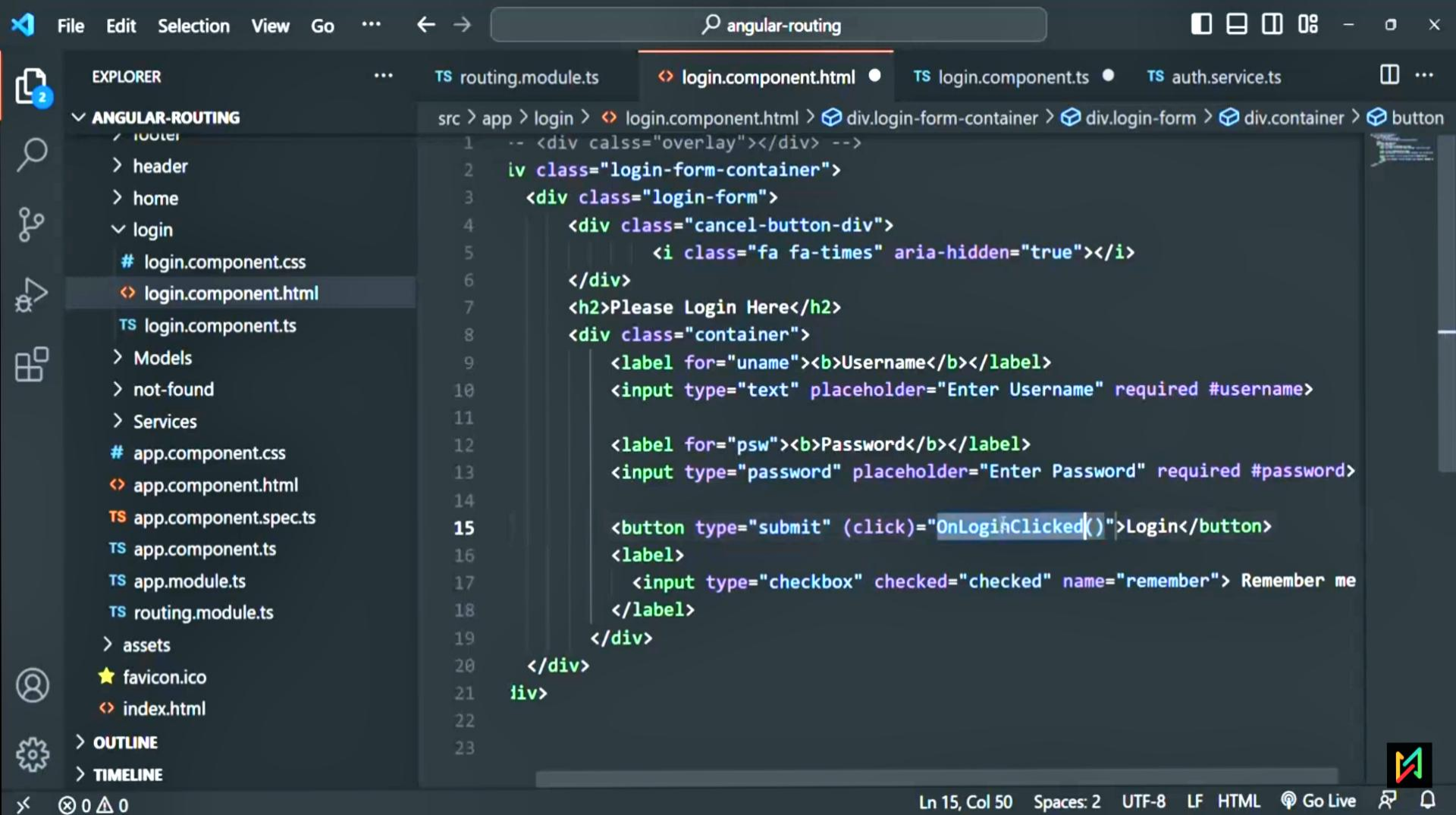
> OUTLINE

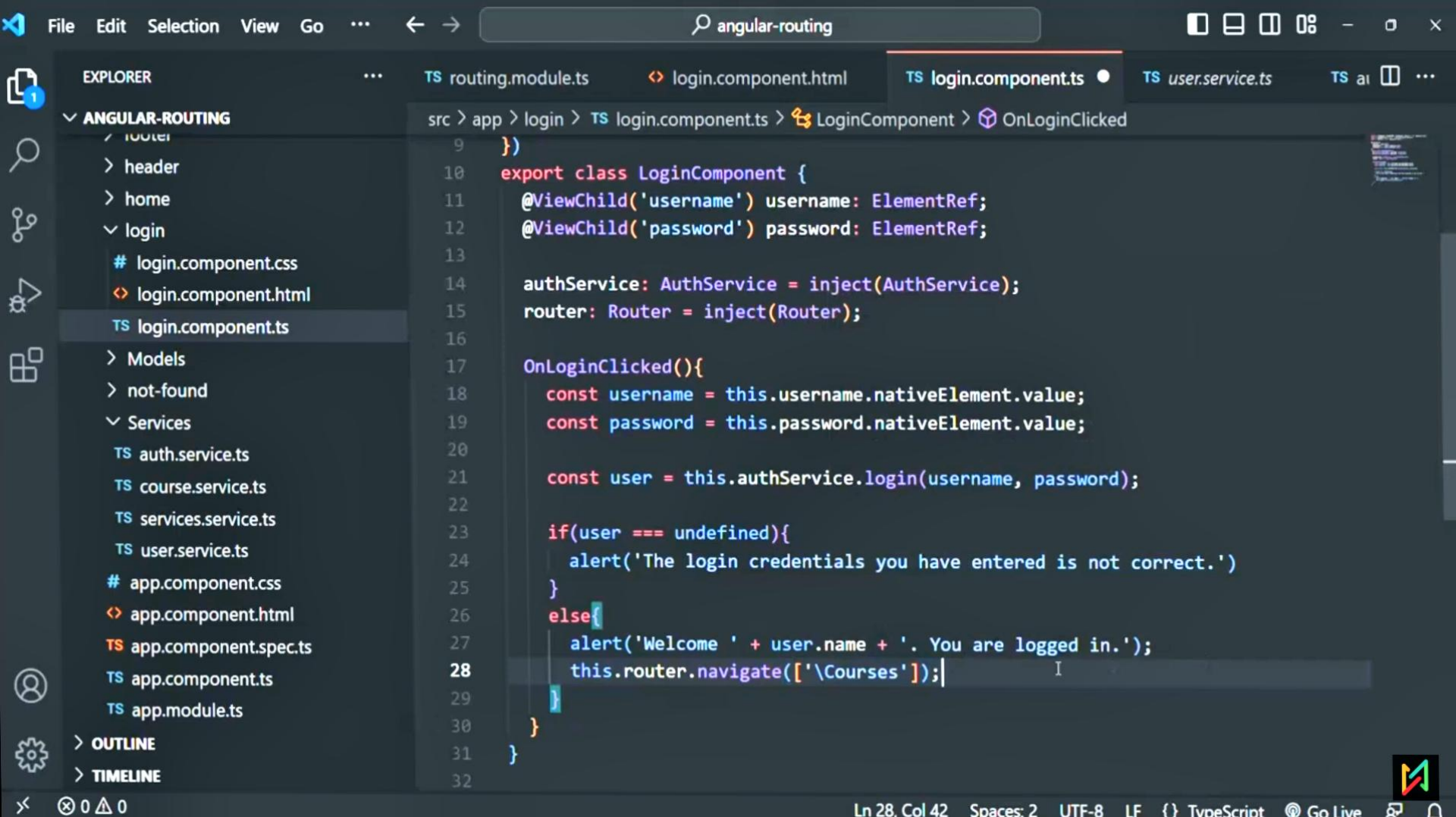
> TIMELINE

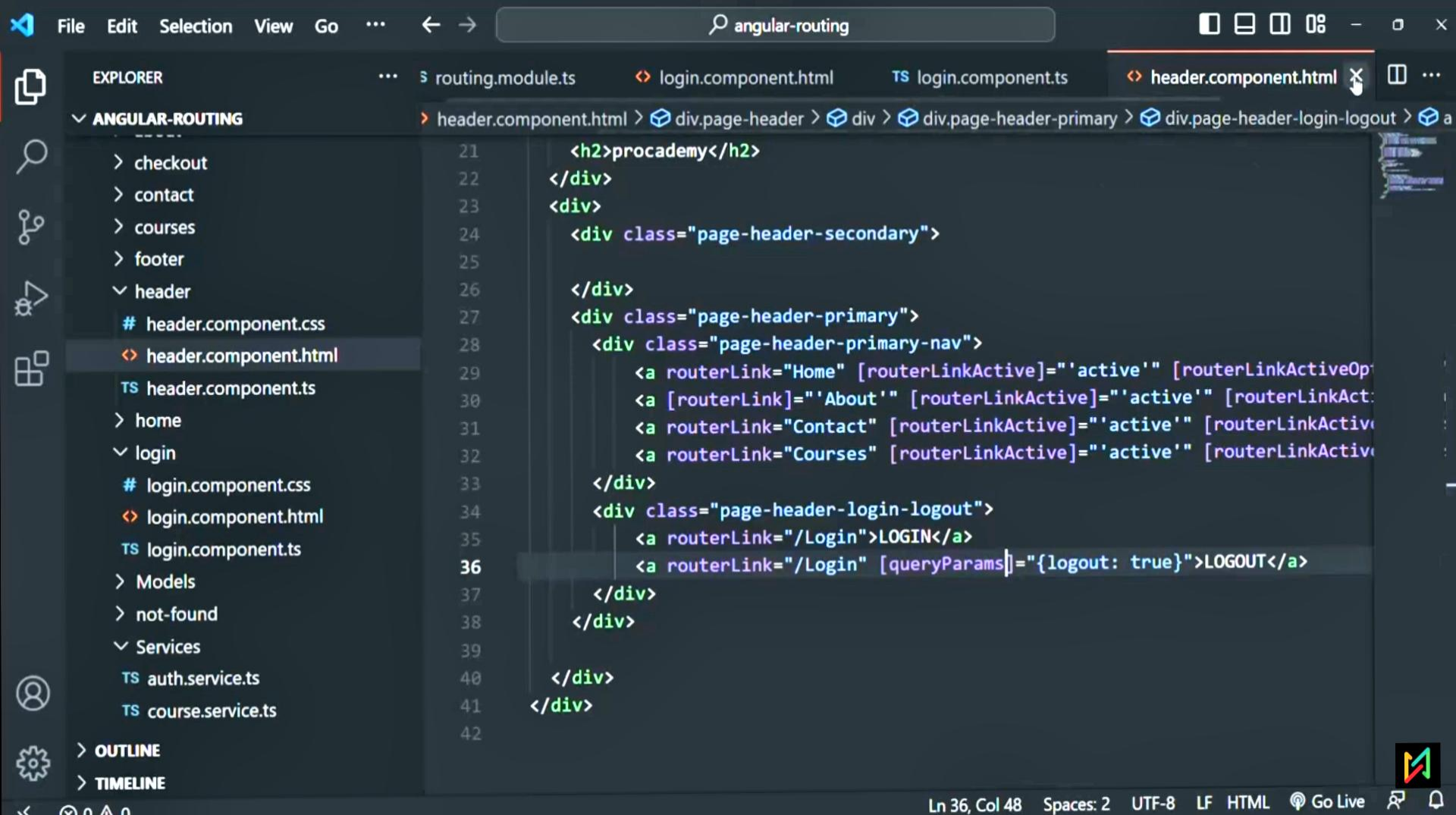
TS routing.module.ts 1 TS user.service.ts TS auth.service.ts

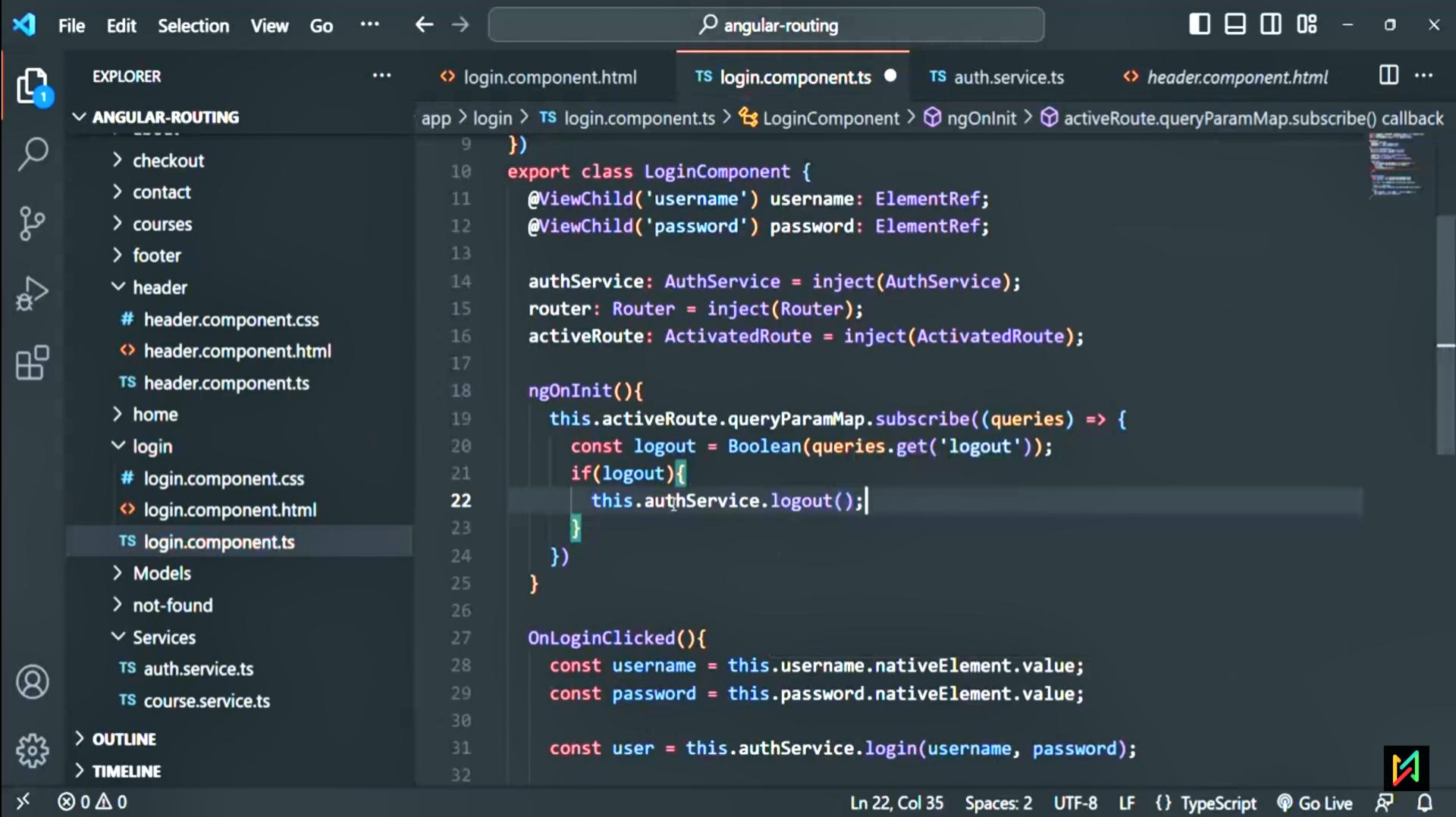
src > app > TS routing.module.ts > routes > component

```
3 import { HomeComponent } from './home/home.component';  
4 import { ContactComponent } from './contact/contact.component';  
5 import { AboutComponent } from './about/about.component';  
6 import { PopularComponent } from './home/popular/popular.component';  
7 import { CoursesComponent } from './courses/courses.component';  
8 import { CourseDetailComponent } from './courses/course-detail/course-detail.component';  
9 import { NotFoundComponent } from './not-found/not-found.component';  
10 import { LoginComponent } from './login/login.component';  
11  
12  
13 //DEFINE ROUTE  
14 const routes: Routes = [  
15  
16     {path: '', component: HomeComponent},  
17     {path: 'Home', component: HomeComponent},  
18     {path: 'About', component: AboutComponent},  
19     {path: 'Contact', component: ContactComponent},  
20     {path: 'Courses', component: CoursesComponent},  
21     {path: 'Courses', children: [  
22         {path: 'Course/:id', component: CourseDetailComponent},  
23         {path: 'Popular', component: PopularComponent},  
24     ]},  
25     {path: 'Login', component: LoginComponent},  
26     {path: '**', component: NotFoundComponent},
```





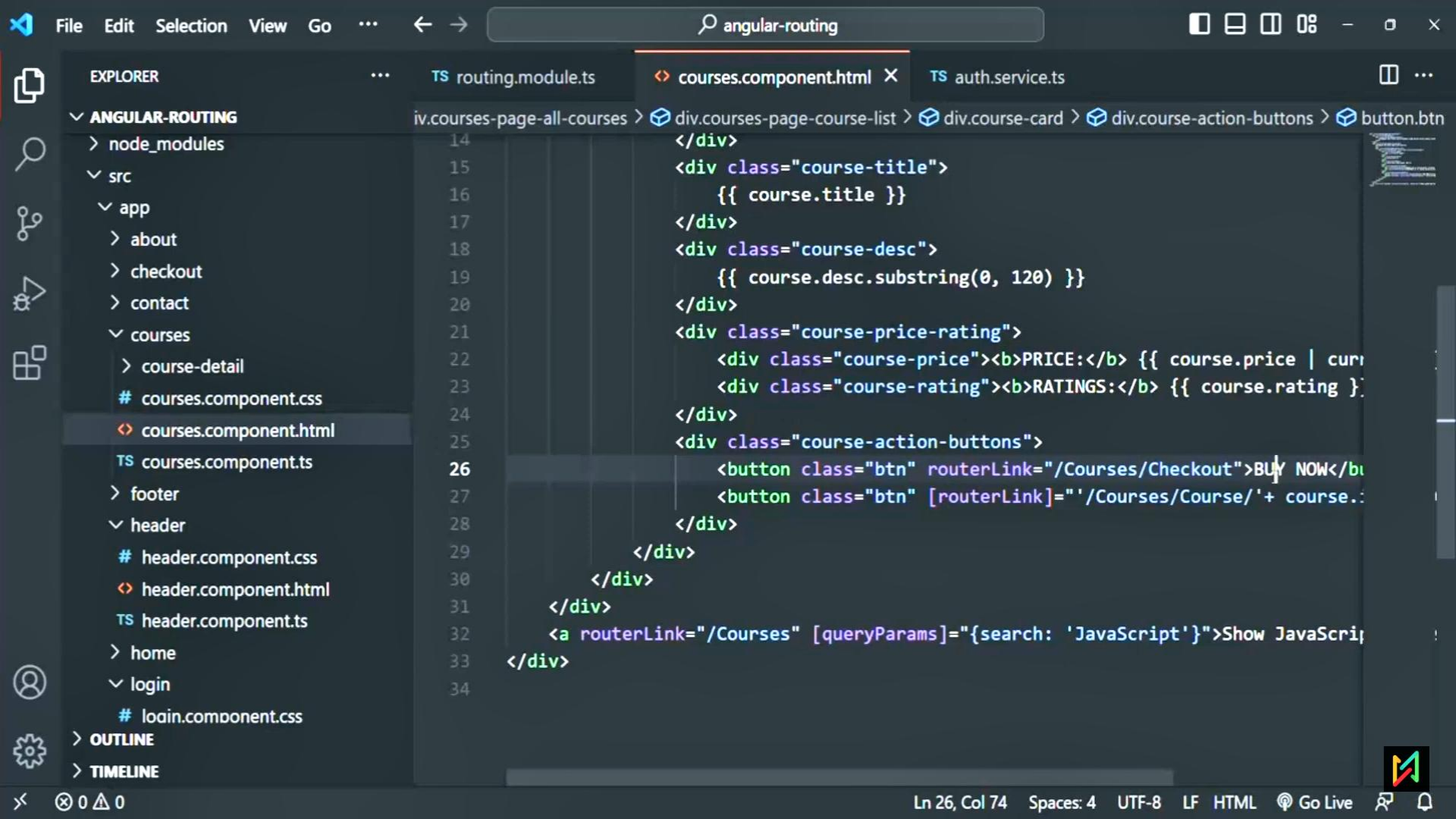




CanActivate Route Guard Use Case

The common use case where we can use a CanActivate route guard is when we want to protect a route from an unauthorized user. For example, let's say there are some routes which can only be accessed by a logged in user, we can use CanActivate guard.

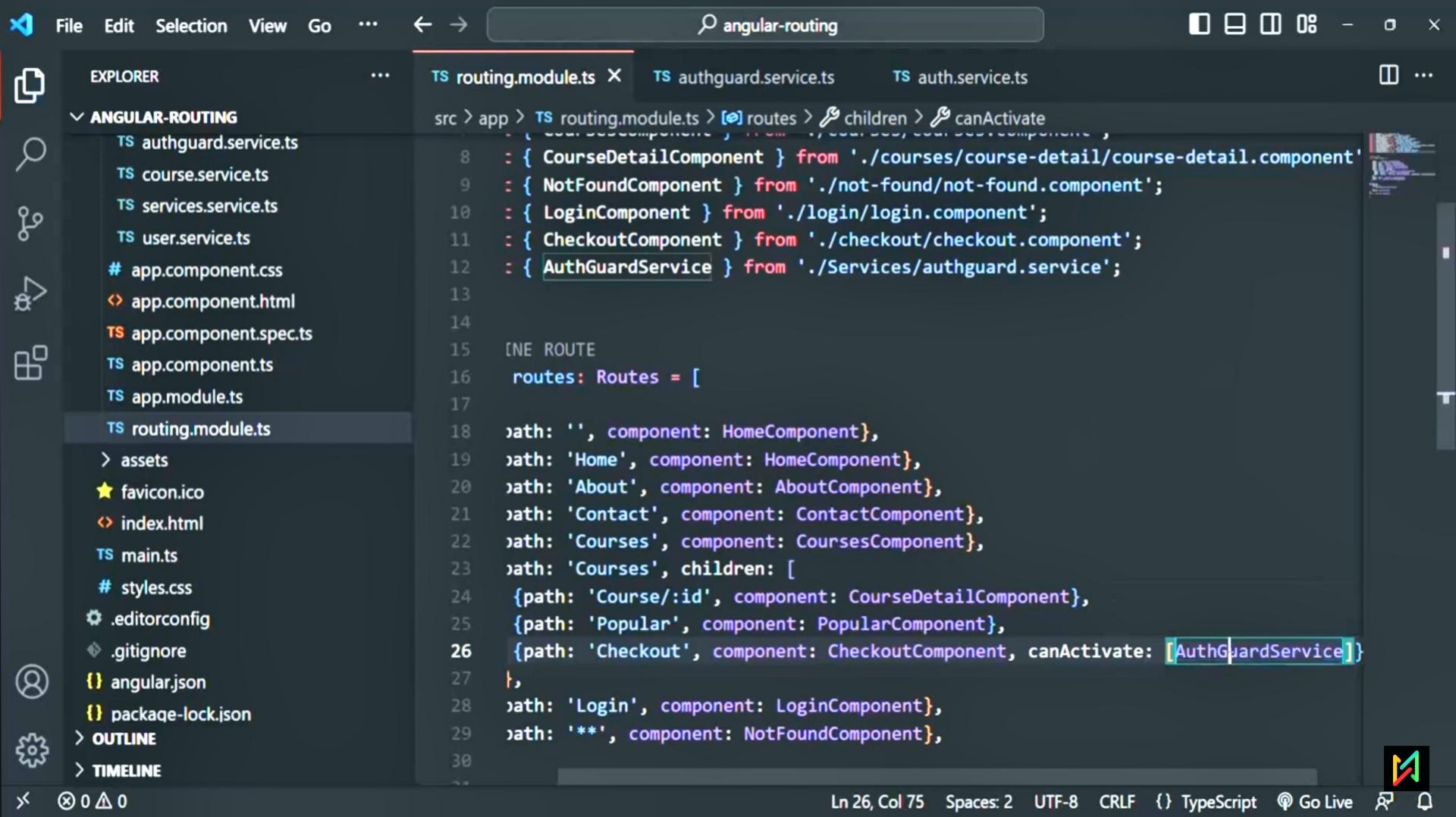


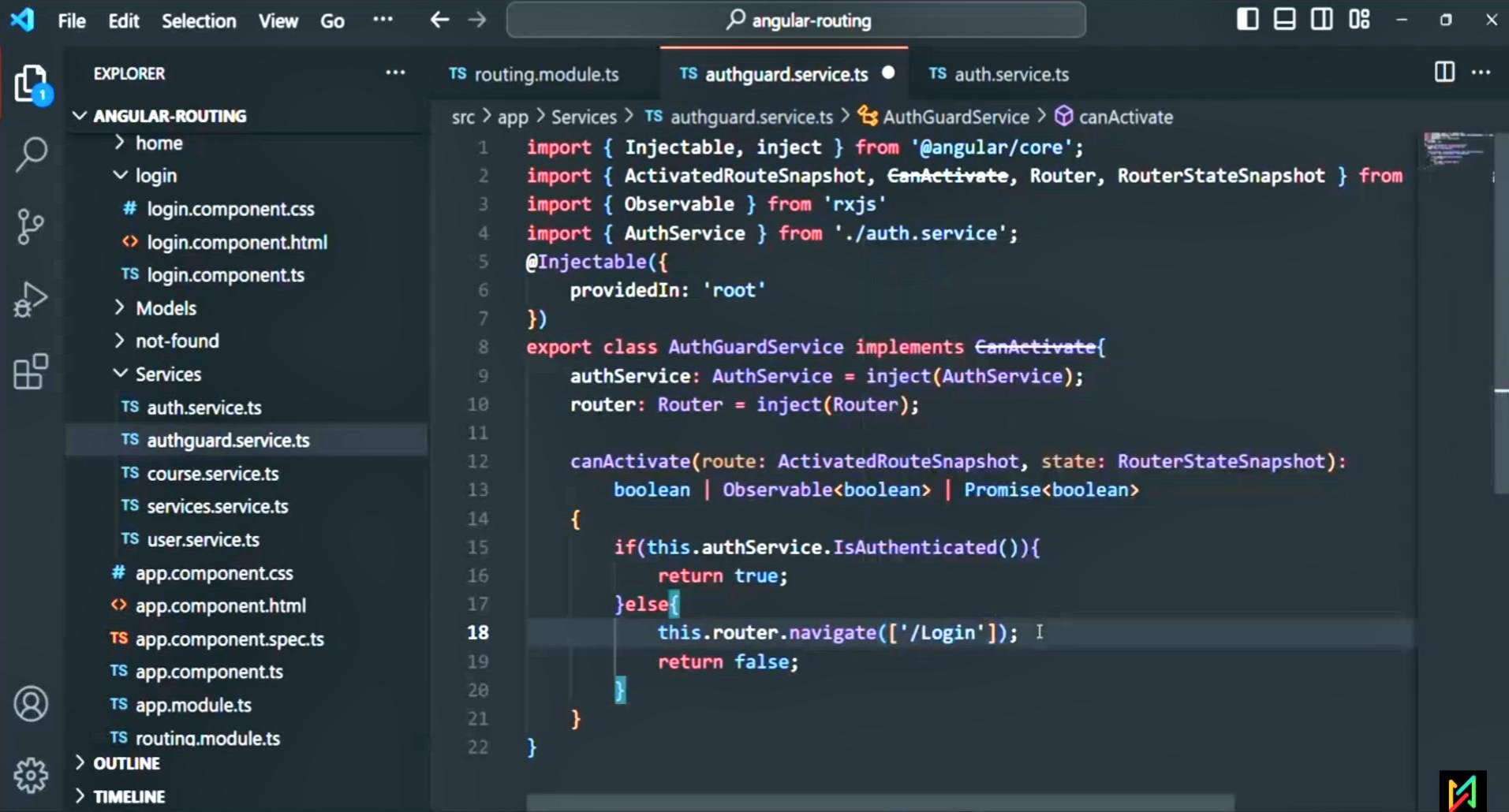


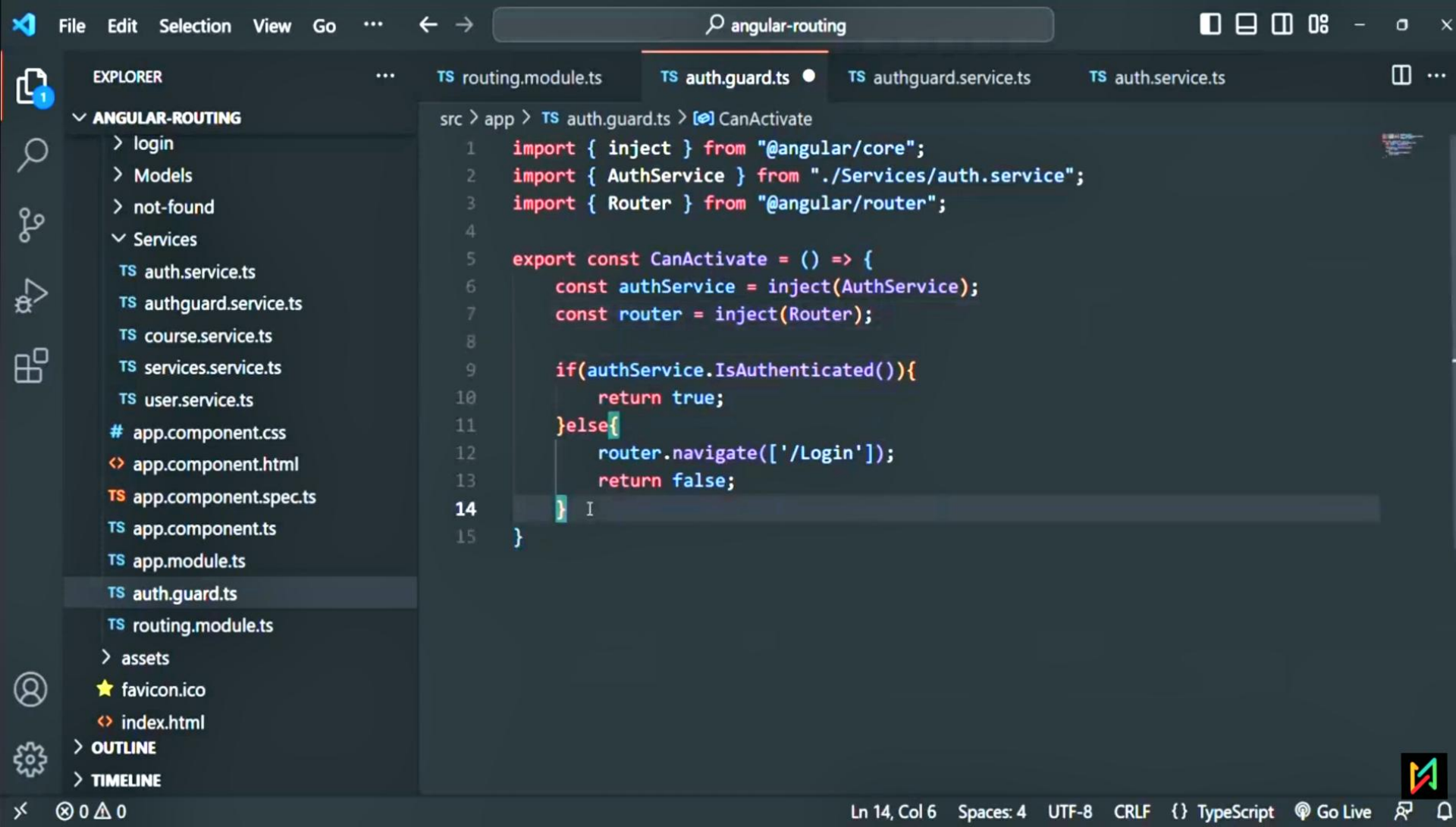
CanActivate Guard: Angular 14 & Lower versions

1. Create a service which inherits from CanActivate interface.
2. Implement CanActivate method provided by CanActivate interface.
3. Return a boolean value from the CanActivate method based on if the route can be activated or not.
4. Assign the service to canActivate property of route object.









EXPLORER

TS routing.module.ts

TS auth.guard.ts

TS authguard.service.ts

TS auth.service.ts

ANGULAR-ROUTING

> login

> Models

> not-found

Services

TS auth.service.ts

TS authguard.service.ts

TS course.service.ts

TS services.service.ts

TS user.service.ts

app.component.css

<> app.component.html

TS app.component.spec.ts

TS app.component.ts

TS app.module.ts

TS auth.guard.ts

TS routing.module.ts

> assets

★ favicon.ico

<> index.html

> OUTLINE

> TIMELINE

src > app > TS auth.guard.ts > CanActivate

```
1 import { inject } from "@angular/core";
2 import { AuthService } from "../Services/auth.service";
3 import { Router } from "@angular/router";
4
5 export const CanActivate = () => {
6     const authService = inject(AuthService);
7     const router = inject(Router);
8
9     if(authService.IsAuthenticated()){
10         return true;
11     }else{
12         router.navigate(['/Login']);
13         return false;
14     }
15 }
```

angular-routing

angular-routing

EXPLOLER

ANGULAR-ROUTING

login

Models

not-found

Services

auth.service.ts

authguard.service.ts

course.service.ts

services.service.ts

user.service.ts

app.component.css

app.component.html

app.component.spec.ts

app.component.ts

app.module.ts

auth.guard.ts

routing.module.ts

assets

favicon.ico

index.html

OUTLINE

TIMELINE

TS routing.module.ts X

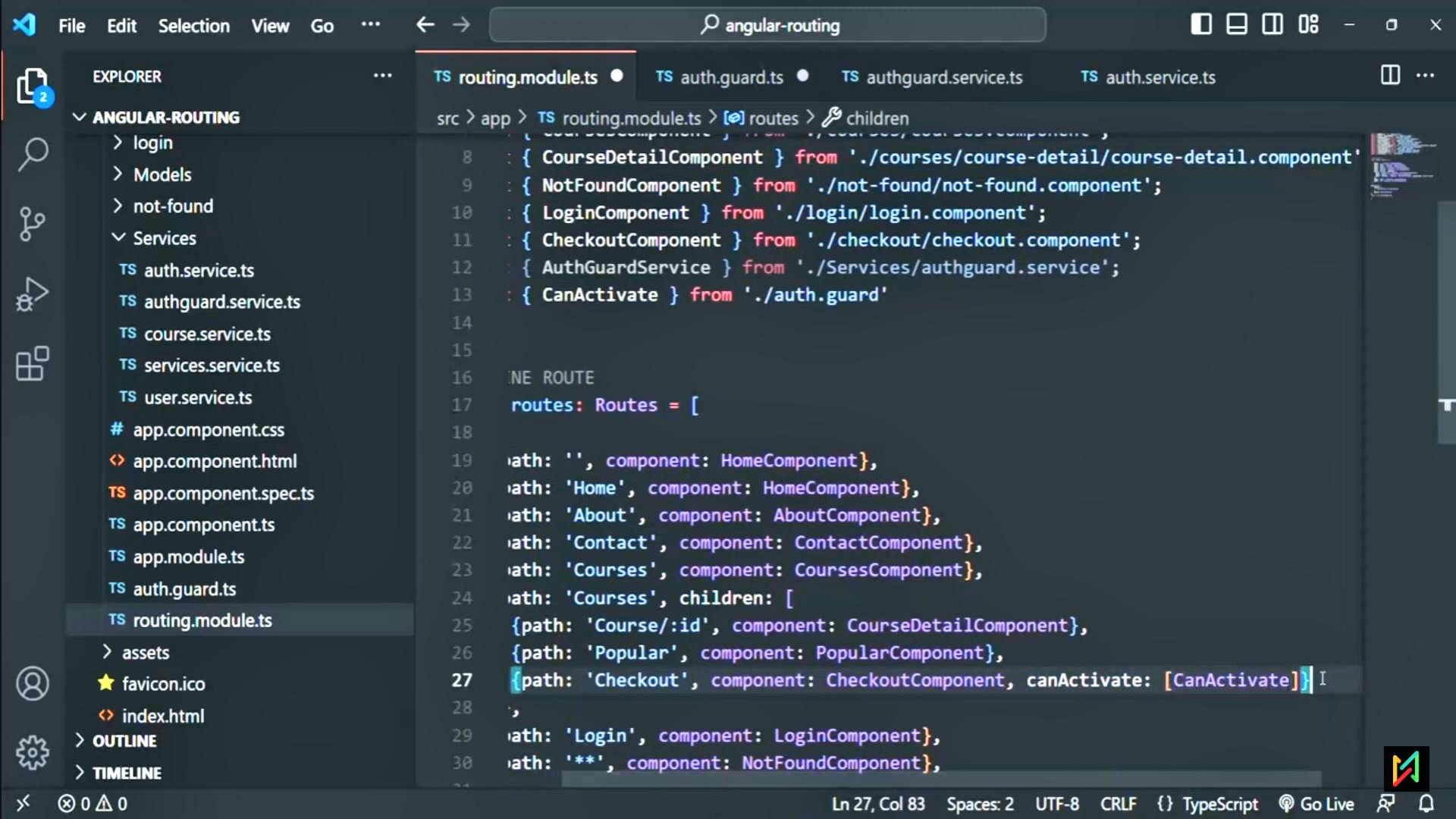
TS auth.guard.ts

TS authguard.service.ts

TS auth.service.ts

src > app > TS routing.module.ts > routes > children > canActivate

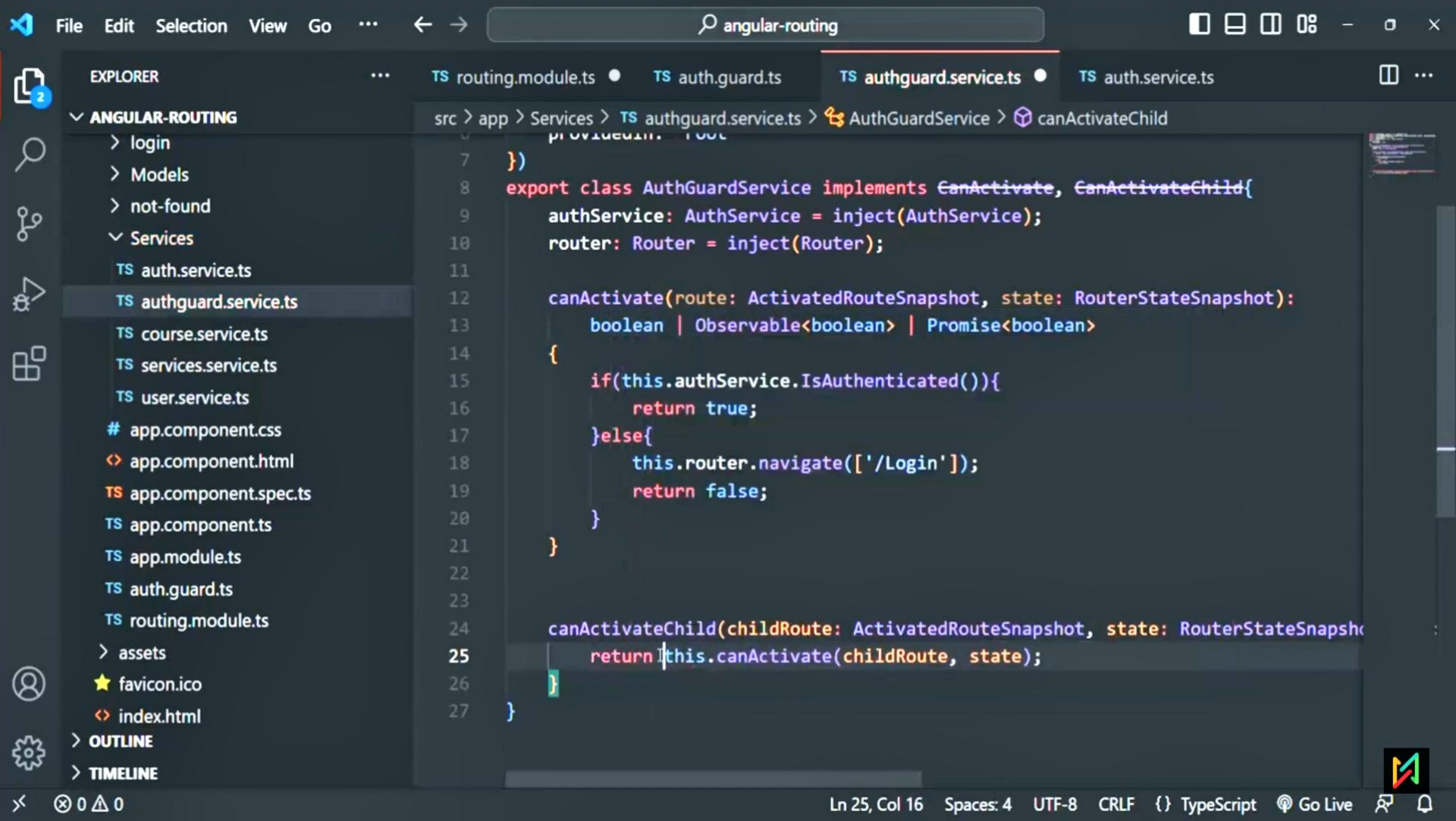
```
8 detailComponent } from './courses/course-detail/course-detail.component';
9 idComponent } from './not-found/not-found.component';
10 component } from './login/login.component';
11 itComponent } from './checkout/checkout.component';
12 ardService } from './Services/authguard.service';
13
14
15
16 routes = [
17
18   component: HomeComponent},
19   ', component: HomeComponent},
20   it', component: AboutComponent},
21   :act', component: ContactComponent},
22   'ses', component: CoursesComponent},
23   'ses', children: [
24     urse/:id', component: CourseDetailComponent},
25     ular', component: PopularComponent},
26     eckout', component: CheckoutComponent, canActivate: [(()) => { return false;}}]
27
28   in', component: LoginComponent},
29   , component: NotFoundComponent},
30
31
```



CanActivateChild Route Guard

The Angular CanActivateChild route guard runs before we navigate to a child route guard. This guard is very similar to CanActivate route guard and it protects a child route from unauthorized access.





EXPLORER

ANGULAR-ROUTING

- > login
- > Models
- > not-found
- > Services
 - TS auth.service.ts
 - TS authguard.service.ts
 - TS course.service.ts
 - TS services.service.ts
 - TS user.service.ts
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts
- TS app.component.ts
- TS app.module.ts
- TS auth.guard.ts
- TS routing.module.ts
- > assets
 - ★ favicon.ico
 - <> index.html

> OUTLINE

> TIMELINE

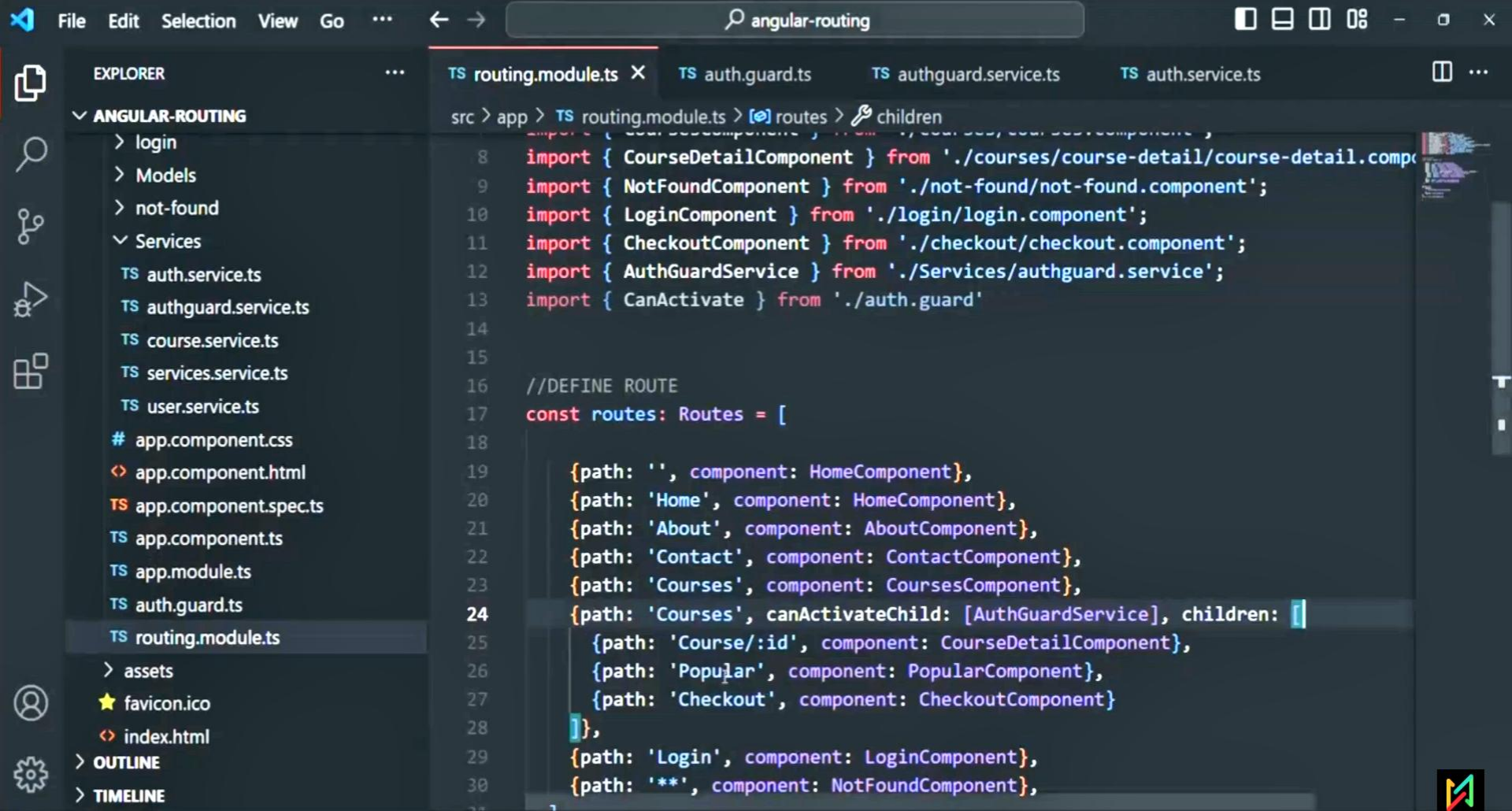
TS routing.module.ts TS auth.guard.ts

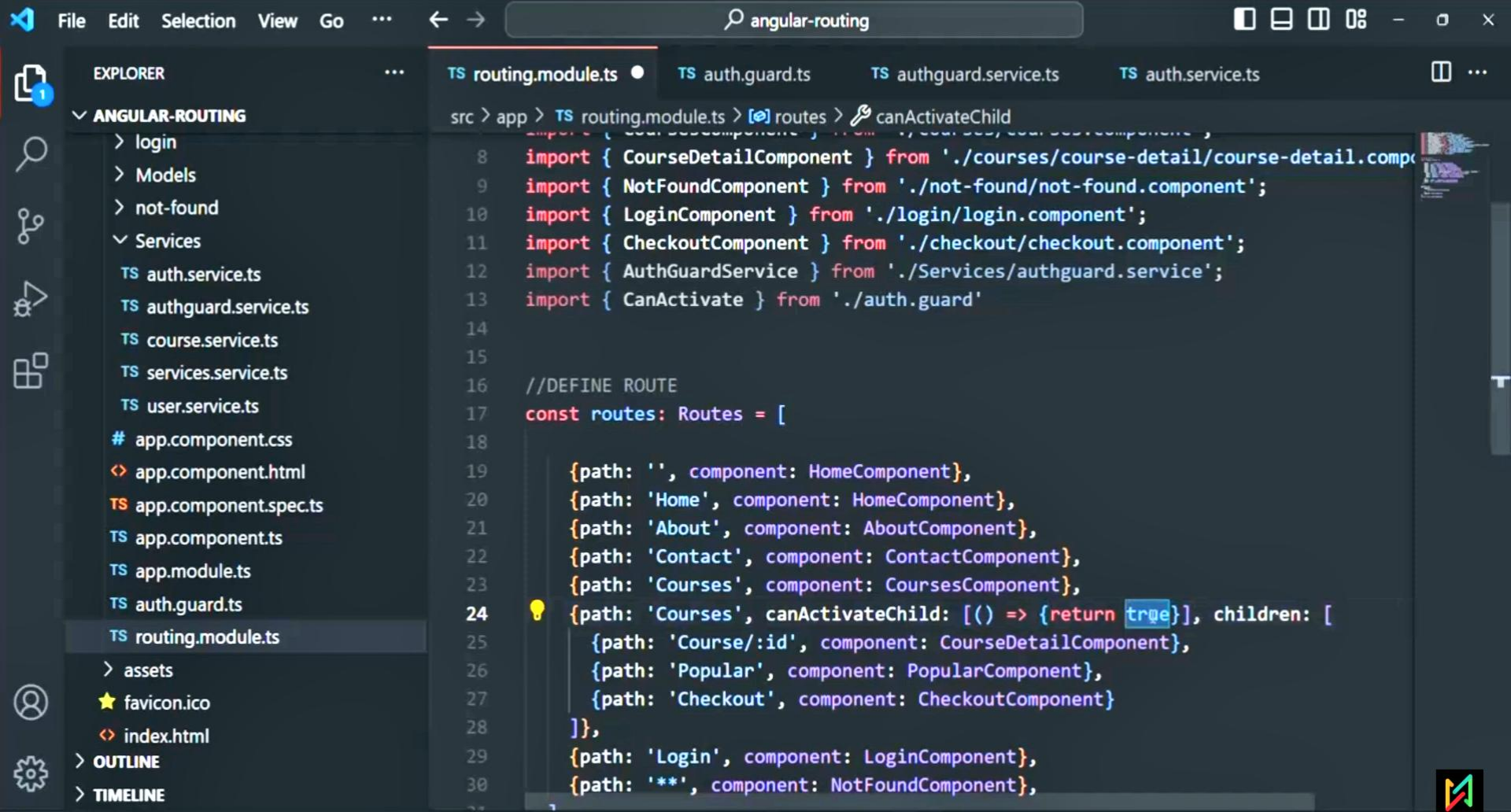
TS authguard.service.ts

TS auth.service.ts

src > app > Services > TS authguard.service.ts > AuthGuardService > canActivateChild

```
6   providedIn: 'root'
7   })
8   export class AuthGuardService implements CanActivate, CanActivateChild{
9       authService: AuthService = inject(AuthService);
10      router: Router = inject(Router);
11
12      canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
13          boolean | Observable<boolean> | Promise<boolean>
14      {
15          if(this.authService.isAuthenticated()){
16              return true;
17          }else{
18              this.router.navigate(['/Login']);
19              return false;
20          }
21      }
22
23      canActivateChild(childRoute: ActivatedRouteSnapshot, state: RouterStateSnapshot):
24          boolean | Observable<boolean> | Promise<boolean>
25      {
26          return this.canActivate(childRoute, state);
27      }
28  }
```





CanDeactivate Route Guard

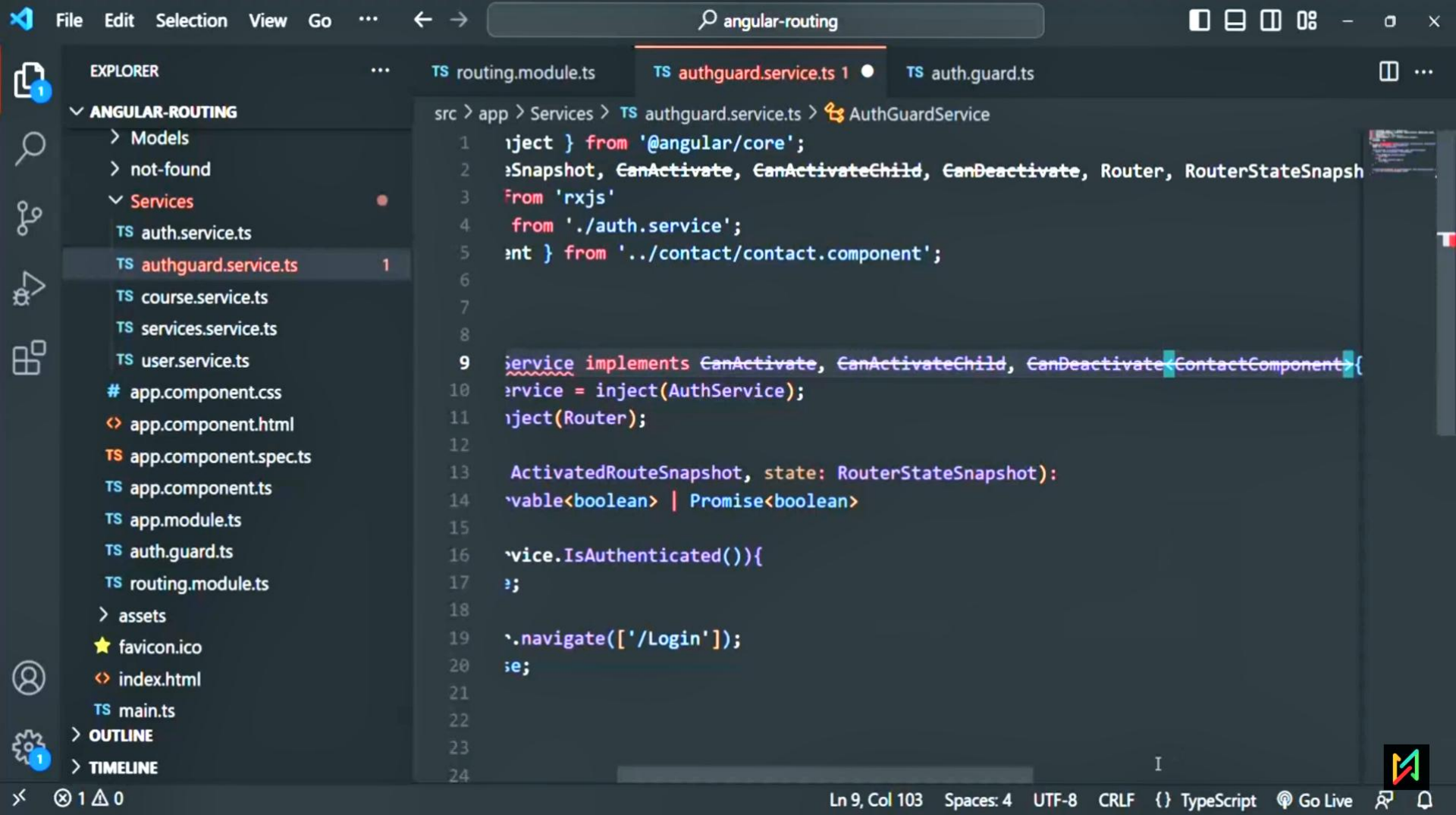
We use CanDeactivate route guard to decide if the user can navigate away from a route or not. This route guard is called whenever we try to navigate away from the current route.



CanDeactivate Route Guard Use Case

The best use case where we can use CanDeactivate route guard is, when the user has filled a form like registration form and tries to navigate away without submitting. In that case, before navigating the user away, we might want to confirm if he wants to submit the data.







EXPLORER

ANGULAR-ROUTING

- Models
- not-found
- Services
 - auth.service.ts
 - authguard.service.ts
 - course.service.ts
 - services.service.ts
 - user.service.ts
- # app.component.css
- <> app.component.html
- TS app.component.spec.ts
- TS app.component.ts
- TS app.module.ts
- TS auth.guard.ts
- TS routing.module.ts
- assets
- ★ favicon.ico
- <> index.html
- TS main.ts
- OUTLINE
- TIMELINE

TS routing.module.ts • TS authguard.service.ts • TS auth.guard.ts

src > app > TS routing.module.ts > routes > canDeactivate

```
11 import { CheckoutComponent } from './checkout/checkout.component';
12 import { AuthGuardService } from './Services/authguard.service';
13 import { CanActivate, CanActivateChild } from './auth.guard'
14
15
16 @NgModule({
17   routes: Routes = [
18
19     {path: '', component: HomeComponent},
20     {path: 'Home', component: HomeComponent},
21     {path: 'About', component: AboutComponent},
22     {path: 'Contact', component: ContactComponent, canDeactivate: [AuthGuardService]},
23     {path: 'Courses', component: CoursesComponent},
24     {path: 'Courses', canActivateChild: [CanActivateChild], children: [
25       {path: 'Course/:id', component: CourseDetailComponent},
26       {path: 'Popular', component: PopularComponent},
27       {path: 'Checkout', component: CheckoutComponent}
28     ]},
29     {path: 'Login', component: LoginComponent},
30     {path: '**', component: NotFoundComponent},
31
32   ],
33 })
34 export class AppRoutingModule { }
```

